1 — Introduction

Machine Learning (MPA-MLR)

Tomáš Vičar
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

Course outline — lectures and labs

Lectures

1. Introduction, optimization
2. Performance evaluation, preprocessing
3. Feature selection, dimensionality reduction
4. Vibecoding — how coding agents work
5. Linear models and kernels
6. Decision trees and ensembles
7. Artificial neural networks
8. Convolutional networks 1: basics
9. Convolutional networks 2: special blocks
10. Other networks: architectures, applications
11. Transformers
12. Generative models, wrap-up

Computer labs (Python)

1. Introduction, libraries, nearest neighbour
2. k-NN as an object, metrics, splitting
3. Feature selection
4. Linear, ridge and kernel regression
5. Coding agents and Git
6. SVM, decision trees, random forest
7. Neural network from scratch
8. PyTorch
9. Deep-learning showcase with an agent
10. Practical application on MetaCentrum
11. Transformers
12. Project

Course evaluation

- Everything about the assessment — the semester tests, the project, the credit and the oral exam — is explained **in the first computer lab**.
- The rules are also on the course web: https://tomasvicar.github.io/MLR-public/organisation.html.

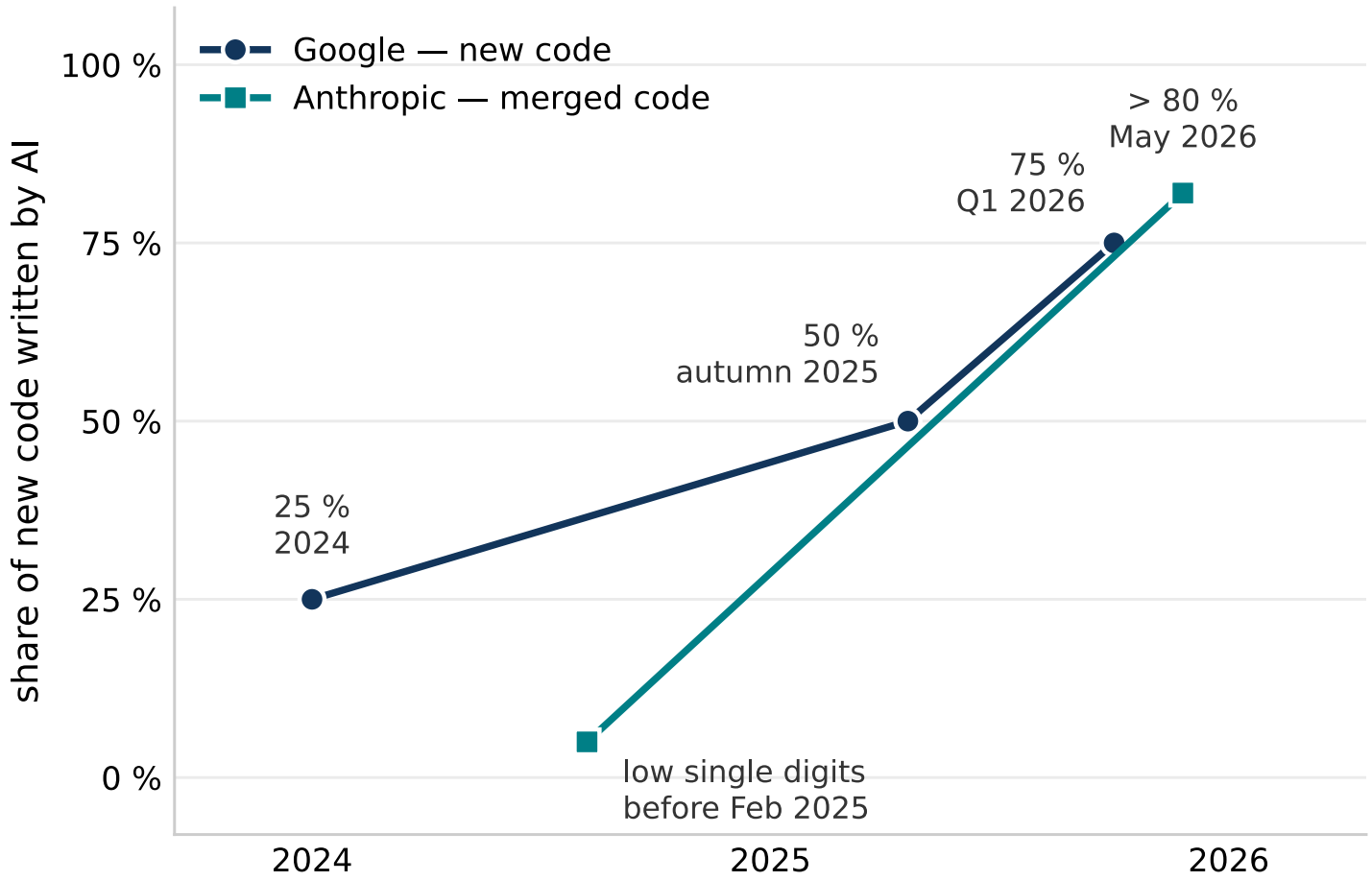
How we code now

> Most new code in the industry is now written by AI agents and only checked by people. What does that mean for a machine-learning course?

Who writes the code today

- The code is written by a **coding agent** (Claude Code, Codex, …). The human **specifies the task and checks the result**.
- A year ago: writing code with AI suggestions. Today: reviewing AI-written code. **It will keep changing this fast.**
- Our own research at the department works the same way.

What the companies report

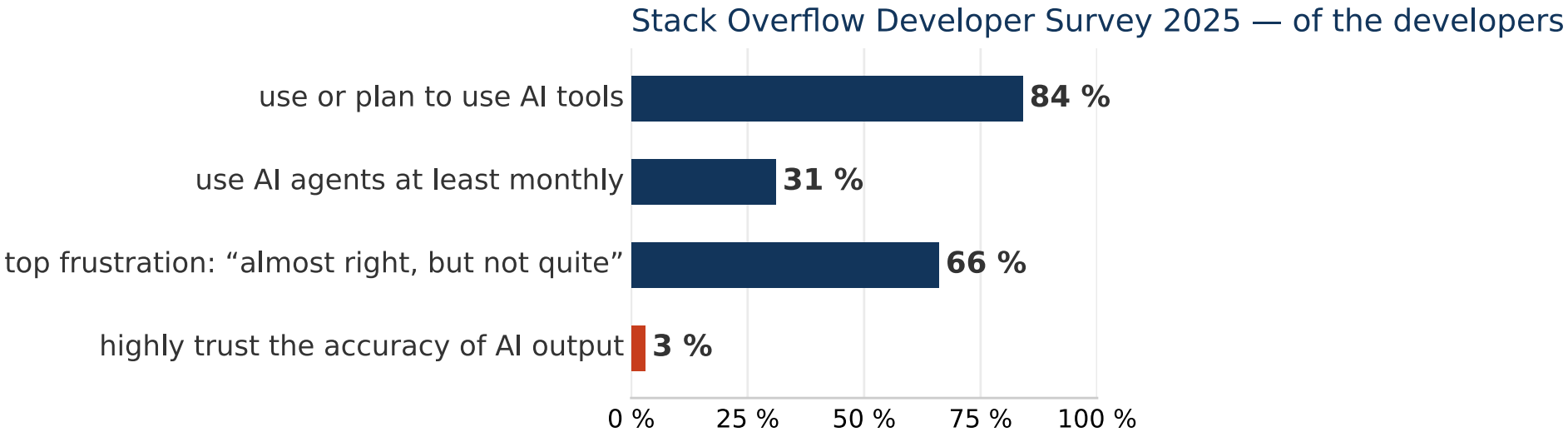


- **Google:** 75 % of new code is AI-generated (25 % in 2024, 50 % in autumn 2025).
- **Anthropic:** more than 80 % of merged code is written by Claude; low single digits before February 2025.

Google: Alphabet Q1 2026 earnings call (Semafor, 24 April 2026). Anthropic: *When AI builds itself*, May 2026.

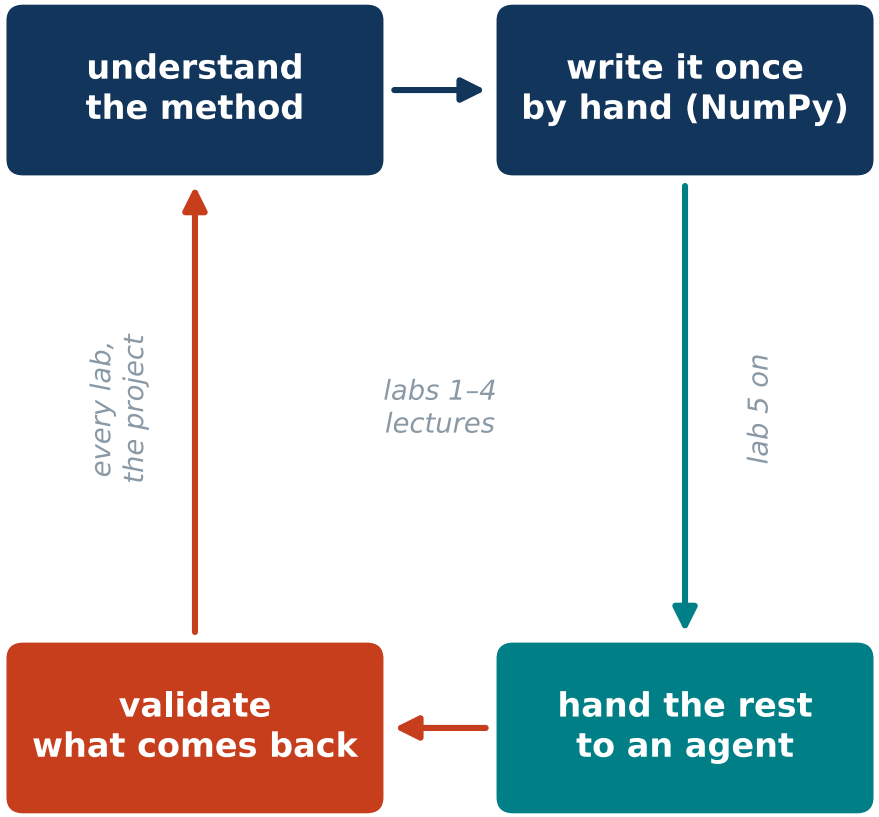
What does that mean for this course?

- Nobody knows which skills will be needed in five years — **so it is hard to decide what to teach.**
- What will not go away: **understanding the methods.** It lets you tell an agent what to build and notice when a result cannot be right.
- Somebody has to tell *almost right* from *right*.



So why do we still write Python by hand?

- **To understand the principle, not to ship it:** k-NN, forward selection, backpropagation — each written once, in NumPy.
- **More by hand:** a distance, a metric, one split of a tree — on paper first, then in code.
- **One lecture and one lab on vibecoding**, then agents in the later labs and the project.
- **Reading code you did not write:** find what is wrong with a pipeline — the main job once an agent writes it.



Outline

1. **Definitions** — what is AI, ML, DL and how they relate
2. **Branches of Machine Learning** — supervised, unsupervised, reinforcement, self-supervised
3. **Optimization** — the language in which almost every ML method is written
4. *Appendix, after the quiz:* **Recap of the simplest algorithms** — distances, k-NN, k-means, UPGMA *(self-study, not presented)*

1 · Definitions

> What exactly do we mean by artificial intelligence, machine learning and deep learning — and how do they fit into each other?

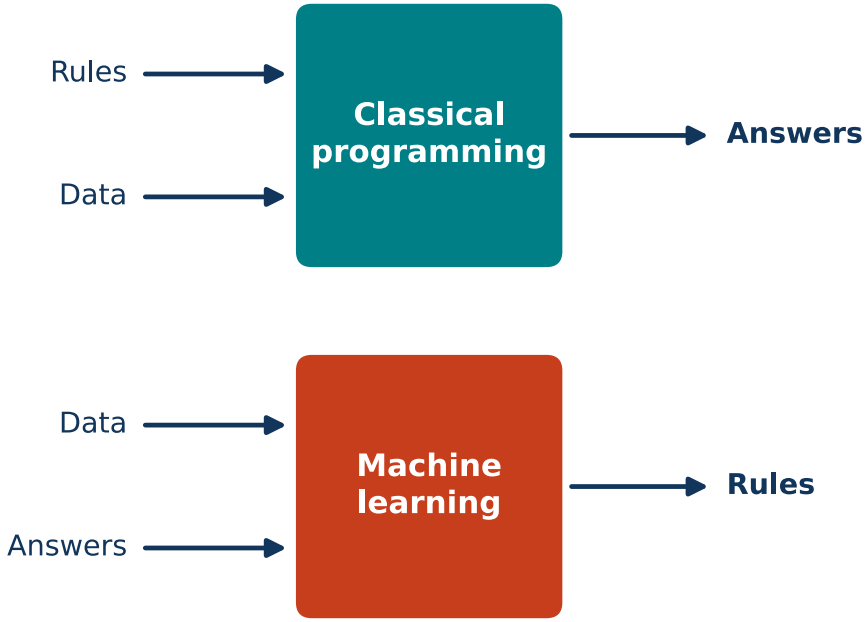
Classical Programming vs. Machine Learning

Classical Programming

- Human provides **rules** + **data**, computer produces **answers**.
- Sorting, taxes, FIR filtering.

Machine Learning

- Human provides **data** + **answers**, computer learns the **rules** (model).
- Spam filter, image classification, LLMs.



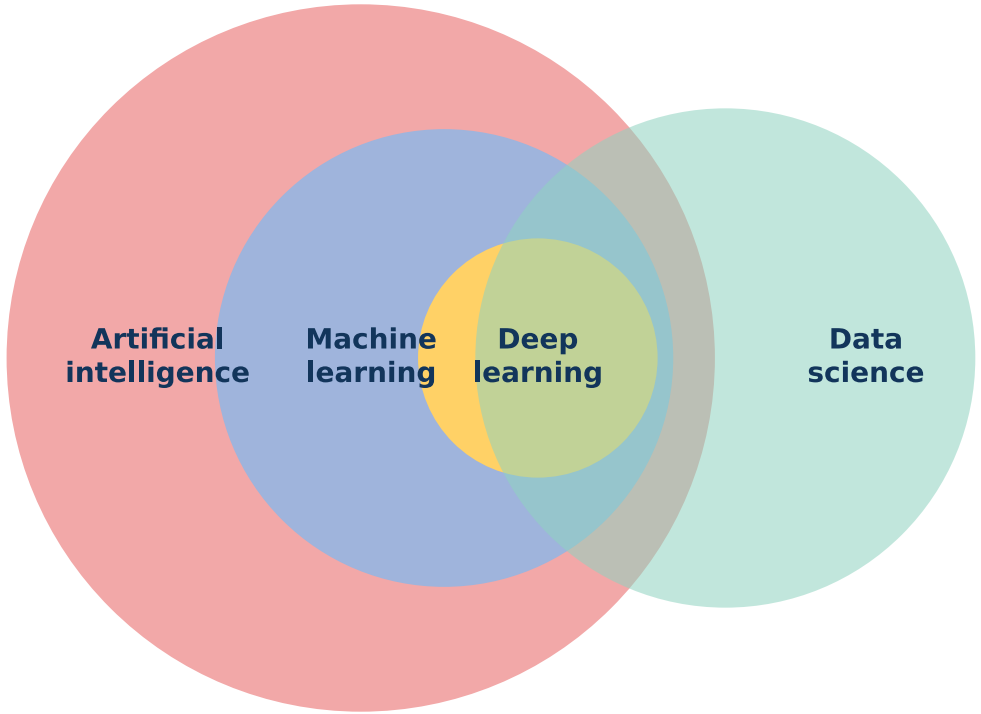
Artificial Intelligence vs. Machine Learning

Artificial Intelligence (AI)

- Systems that perform tasks requiring human intelligence: language, decisions, driving.
- **AI = umbrella term**.

Machine Learning (ML)

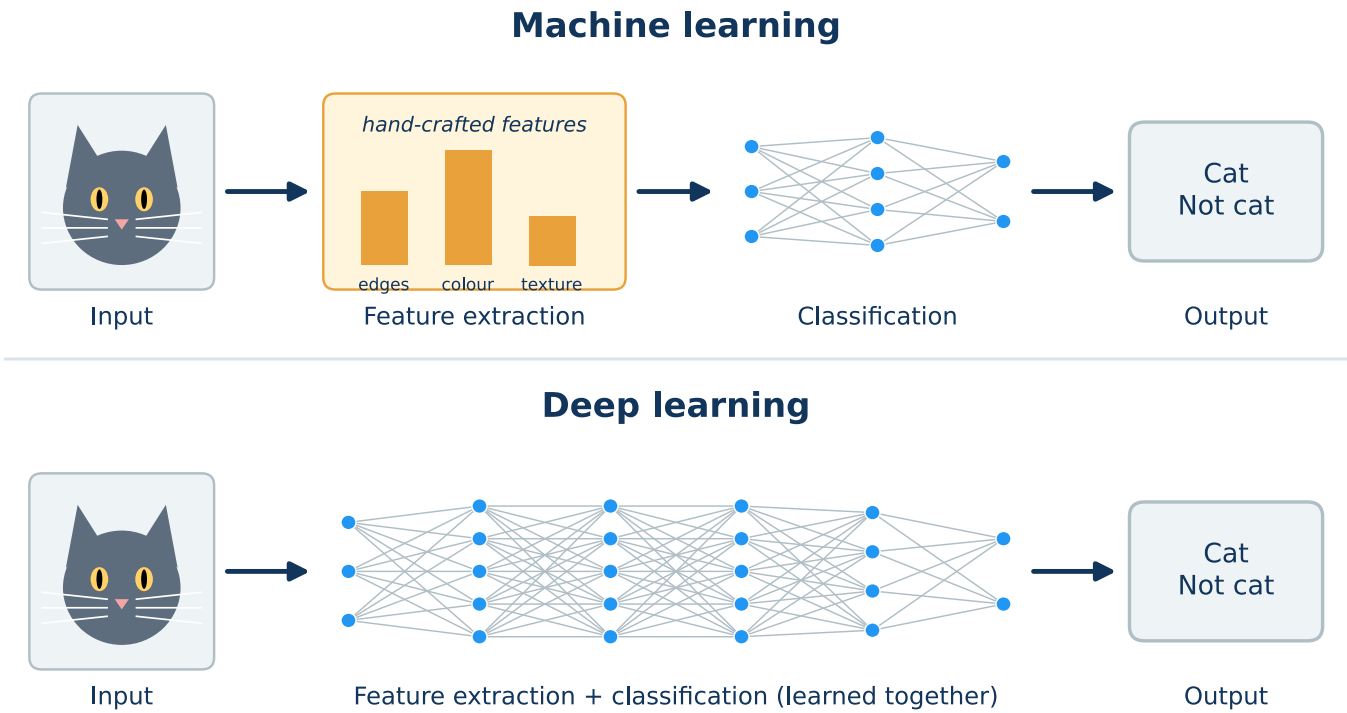
- Learning patterns from data: spam, recommenders, medical images.
- **ML = one way to achieve AI**.



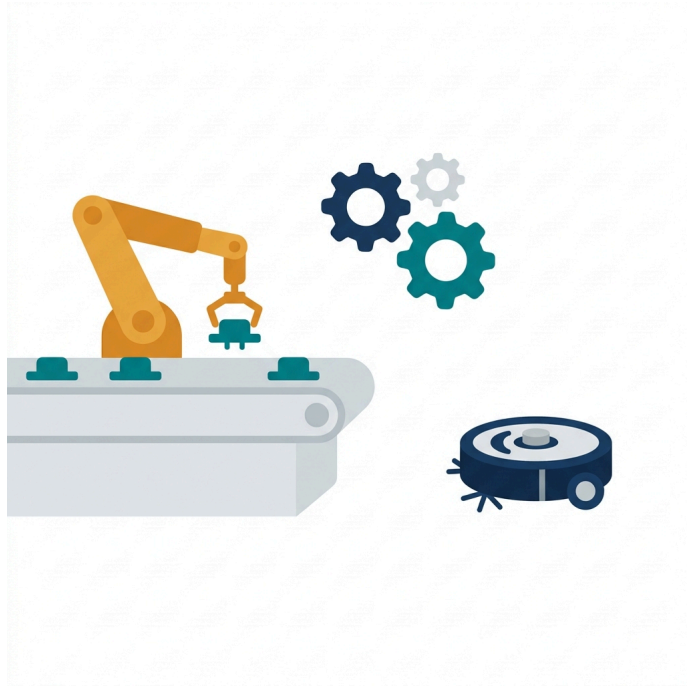
Machine Learning vs. Deep Learning

Deep learning = ML with *neural networks of many layers*: the features are learned **automatically**, at the price of big data and compute.

Machine Learning	Deep Learning
Manual features	Automatic features
Small data OK	Needs big data
Traditional algorithms	Neural networks



Related fields: data science, computer vision, robotics



Data science — statistics + ML + computer science; **insight from data**. ML is one of its tools.

Computer vision — *information from images and video*: detection, segmentation, medical imaging. Today mostly deep learning.

Robotics — *building and programming robots* for the real world; AI + vision + control + mechanics.

The Turing test, narrow AI and AGI

Narrow AI (weak AI) — one system, *one task*: a spam filter, a recommender, an image classifier. **Everything in this course is narrow AI.**

Artificial General Intelligence (AGI, strong AI) — a hypothetical system that would *learn and apply knowledge across domains*, any intellectual task a human can do. Still a **theoretical concept**; LLMs come closest.

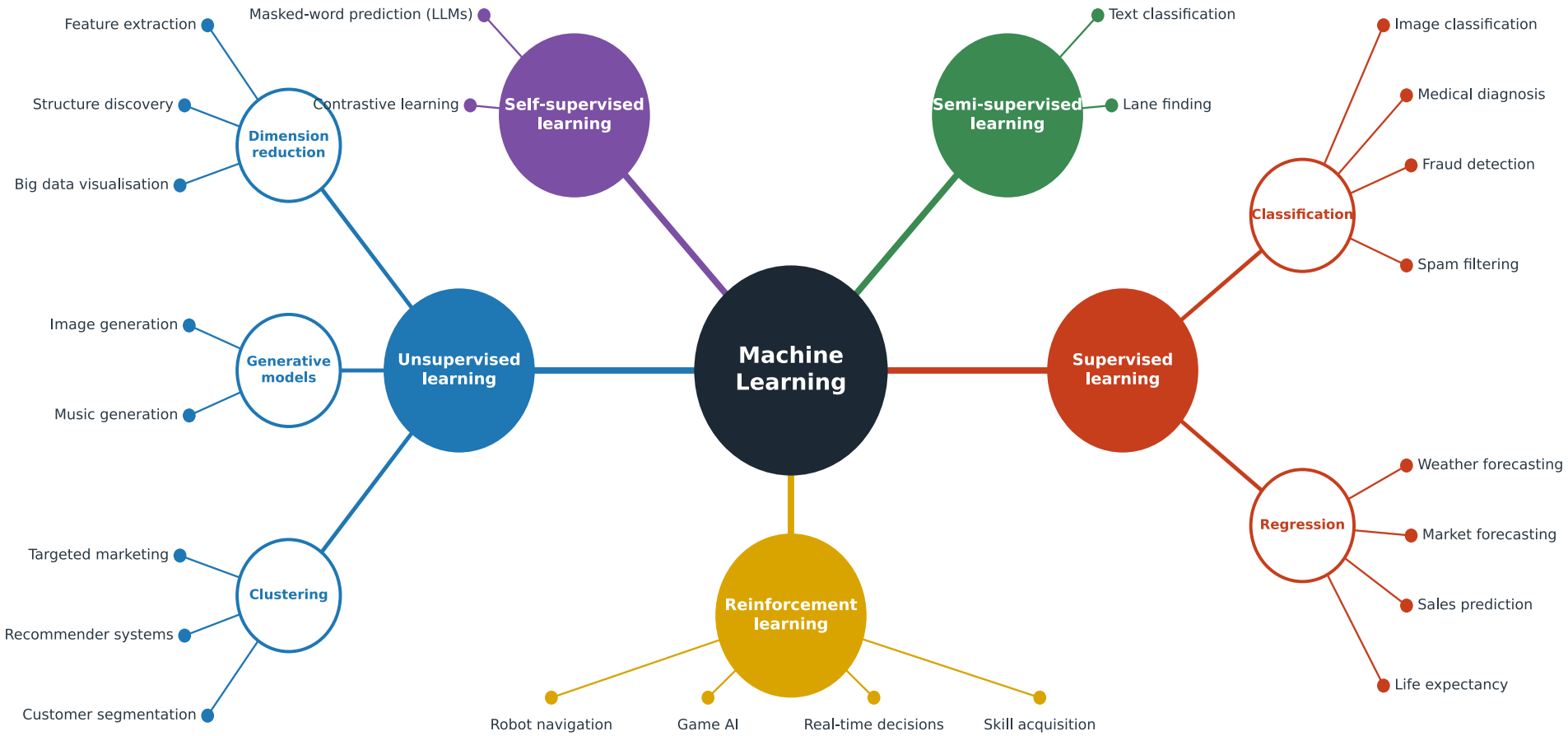
Do machines *think*, or only *simulate* thinking? An open question — this course does not need the answer.

The Turing test (1950): if an evaluator cannot tell the machine's replies from a human's, the machine behaves intelligently. LLMs pass it — the test measures **behaviour, not understanding**.

2 · Branches of Machine Learning

What kind of information do we give the algorithm — labels, nothing at all, or a reward?

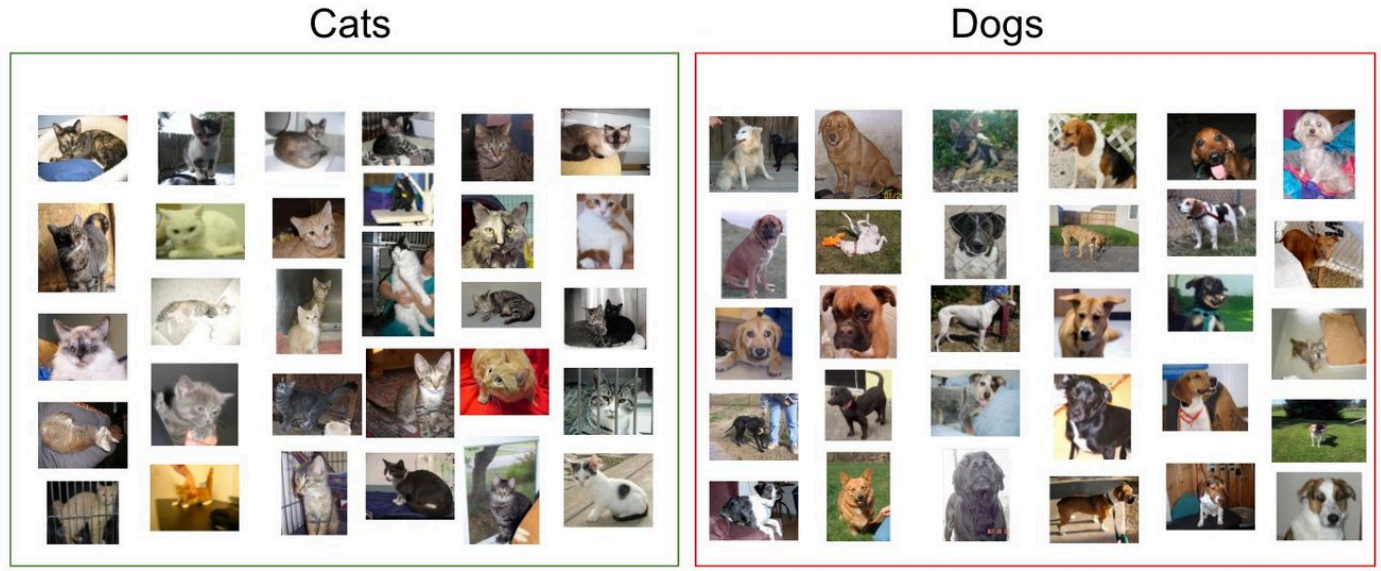
Branches of Machine Learning



Supervised Learning

Definition

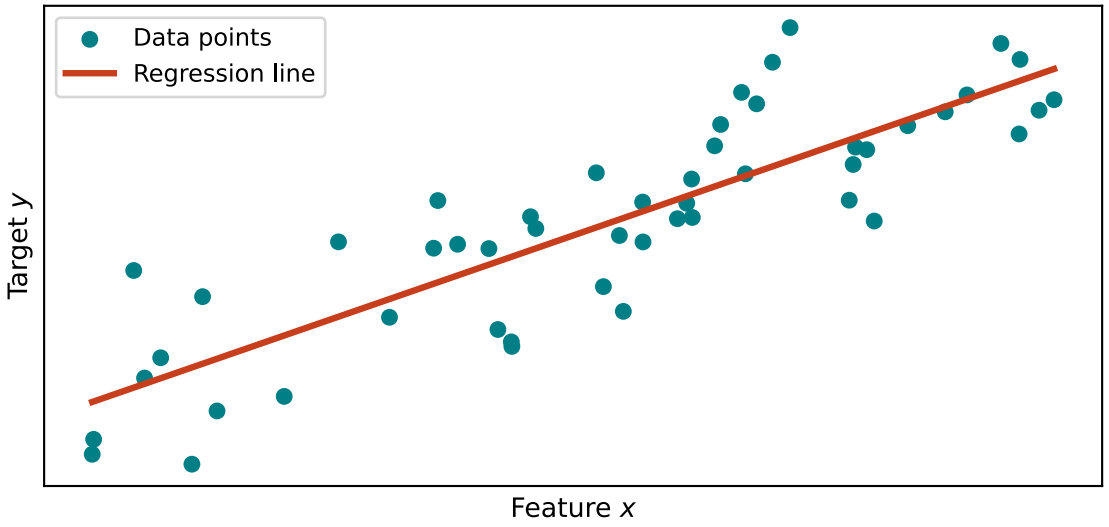
- Learning from *labelled data*: inputs $\mathbf{x}$ with known outputs y.
- Goal: a mapping $f : \mathbf{x} \mapsto y$ that **generalizes** to unseen data.
- Example: cat vs. dog.

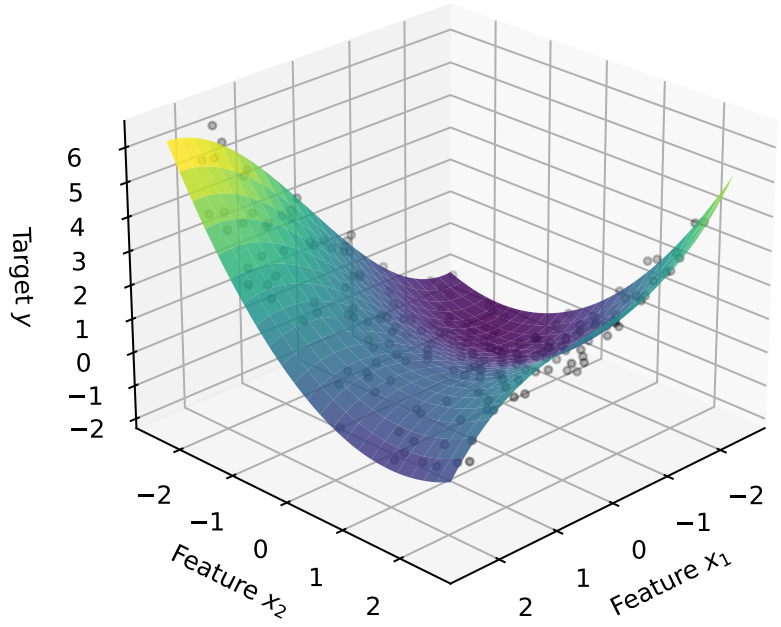


Sample of cats & dogs images from Kaggle Dataset

Regression

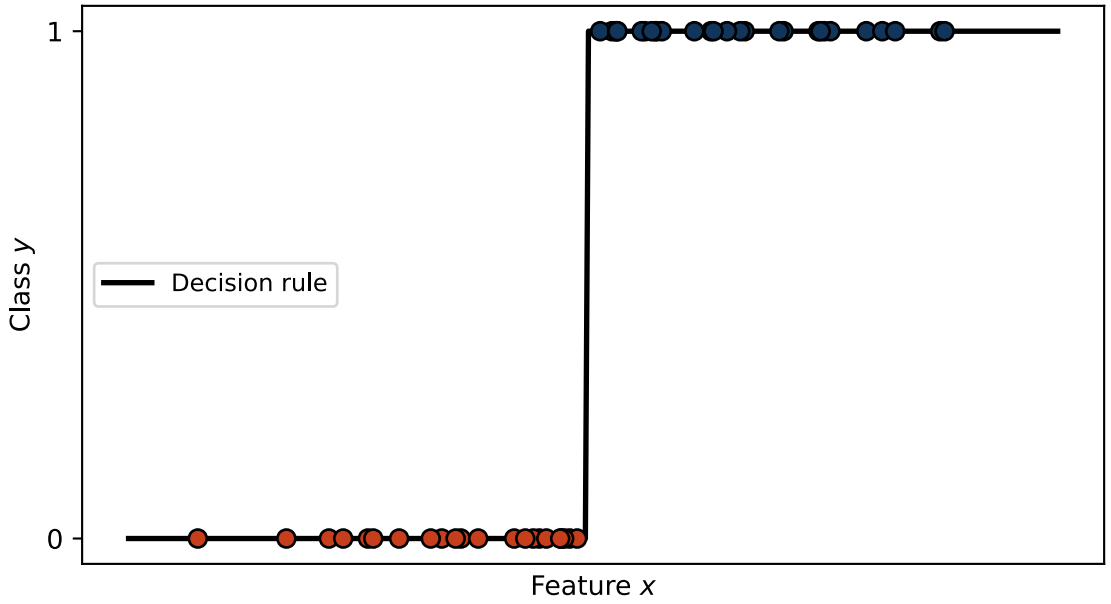
- *Supervised* learning with a **continuous** target y.
- Fit $\hat{y} = f(\mathbf{x})$ to minimize the error between $\hat{y}$ and y.
- Example: house prices, cholesterol level.

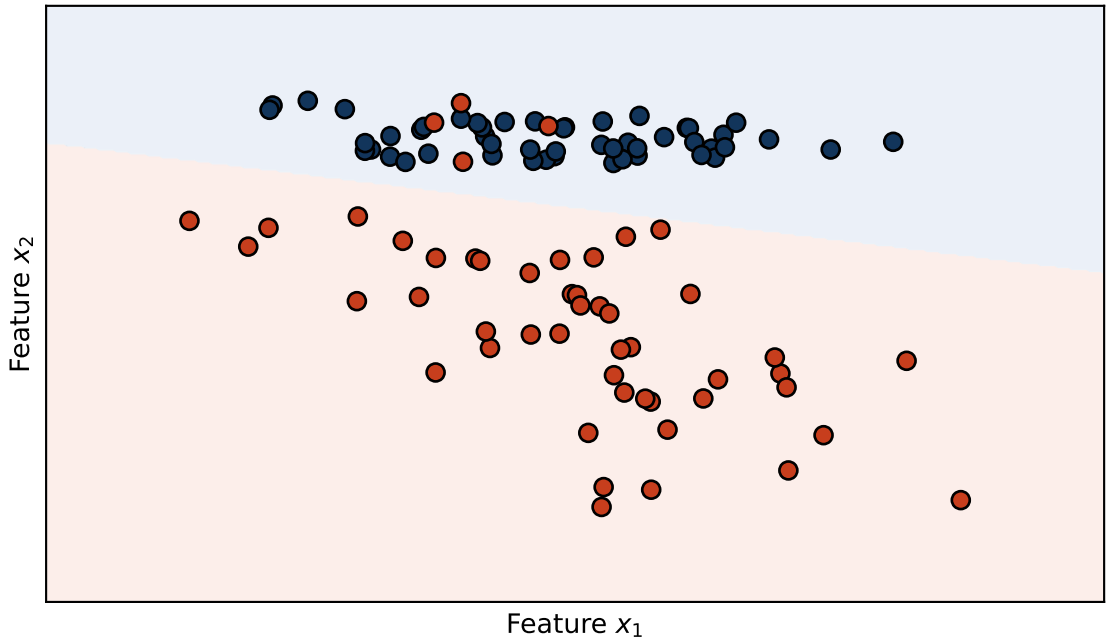




Classification

- *Supervised* learning with a **discrete** target: one of several *classes*.
- Learn a decision function $\hat{y} = f(\mathbf{x})$ that separates the classes.
- Example: **cat vs. dog**, **healthy vs. diseased**.





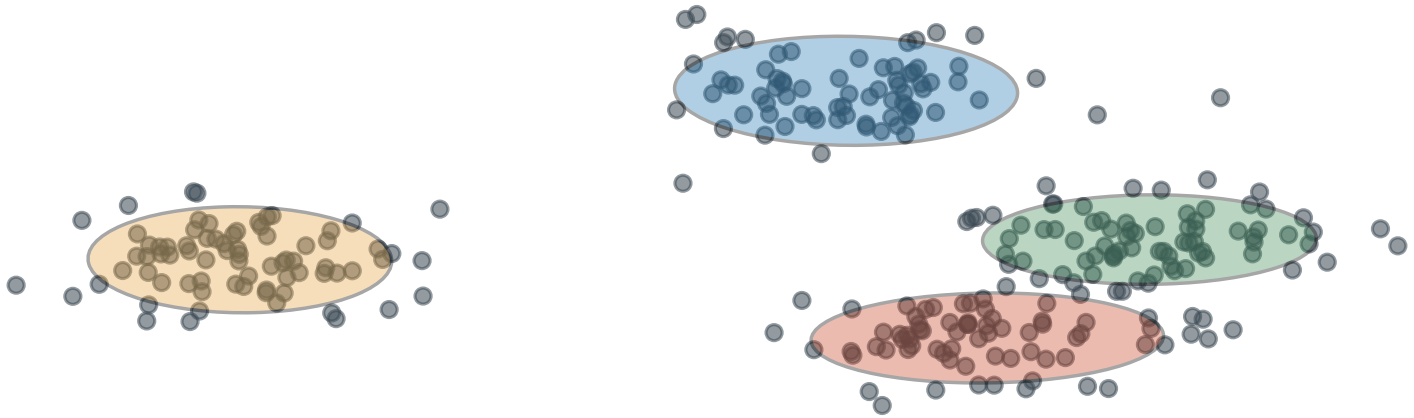
Unsupervised Learning

Definition

- Learning from *unlabelled data*: only $\mathbf{x}$, no y.
- Goal: find **hidden structure** in the data.

Three branches (one slide each)

- **Clustering** — group similar samples.
- **Dimensionality reduction** — fewer features, same structure.
- **Generative models** — learn the distribution, draw new samples.



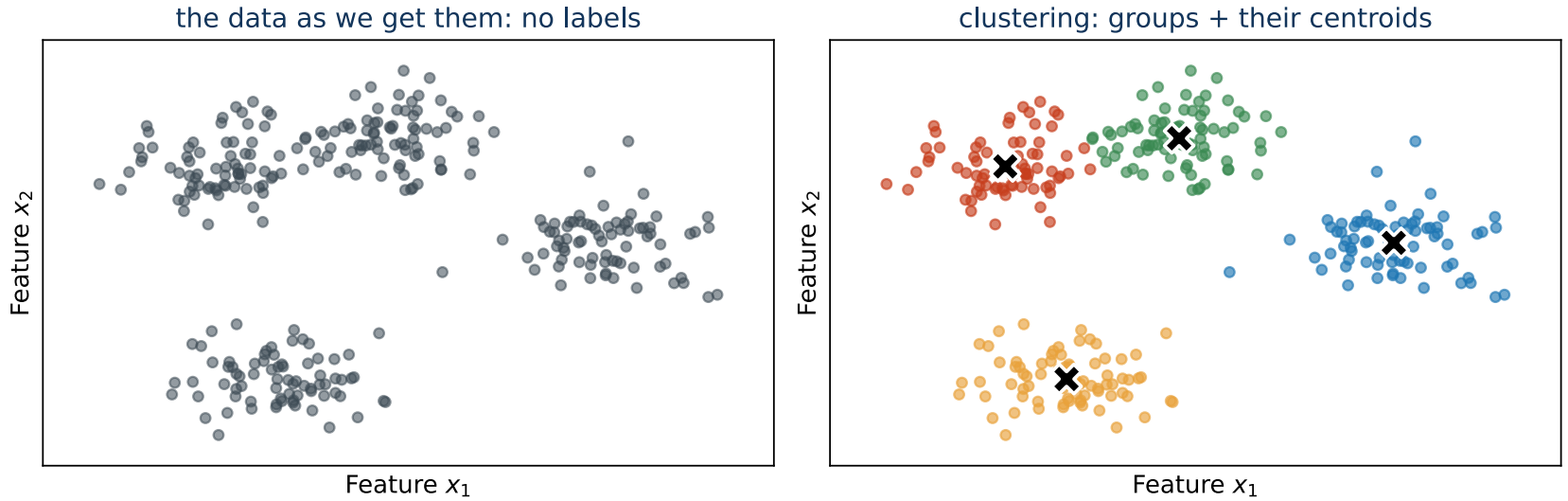
Unsupervised Learning: Clustering

What it is

- Group the samples so that **similar ones end up together**.
- No labels — **nobody says how many groups** or what they mean.
- "Similar" = *close*: everything hangs on the distance.

Typical uses

- Customer segments, cell types.
- Grouping images or documents before annotation.



Two clustering algorithms, **k-means** and **UPGMA**, are recapitulated in the appendix of this lecture.

Unsupervised Learning: Dimensionality Reduction

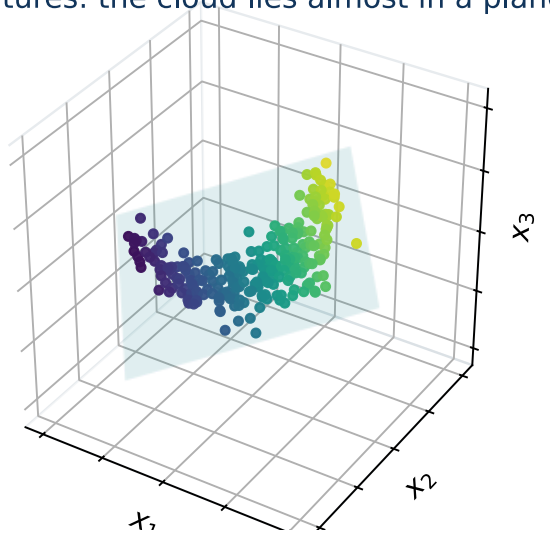
What it is

- Replace many features by a few and keep the **structure**.
- Keep the directions with variation, drop those where the data barely change.

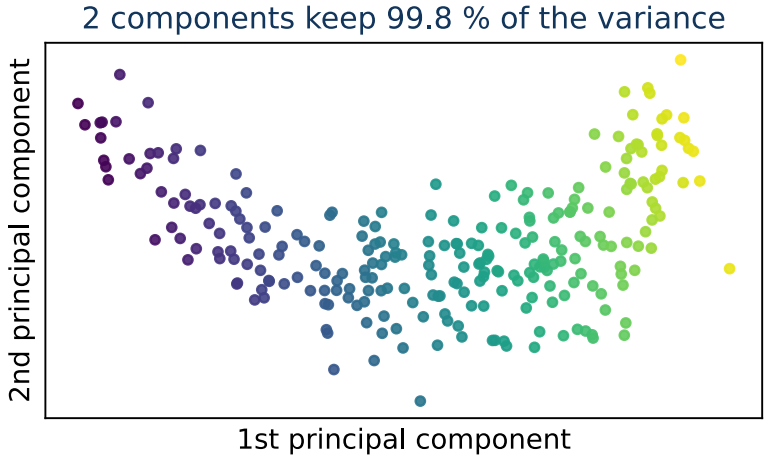
Typical uses

- **Visualisation** — 2 dimensions instead of 200.
- **Denoising**, **compression before a classifier**.

3 features: the cloud lies almost in a plane



2 components keep 99.8 % of the variance



Feature selection, PCA, t-SNE and UMAP are the subject of the whole third lecture.

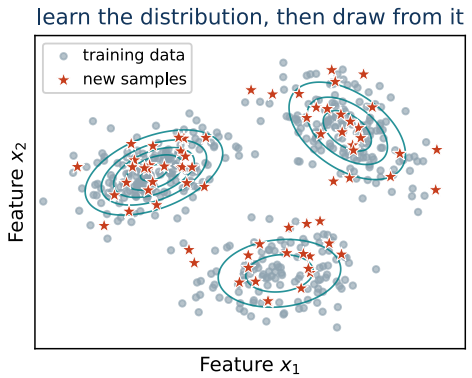
Unsupervised Learning: Generative Models and LLMs

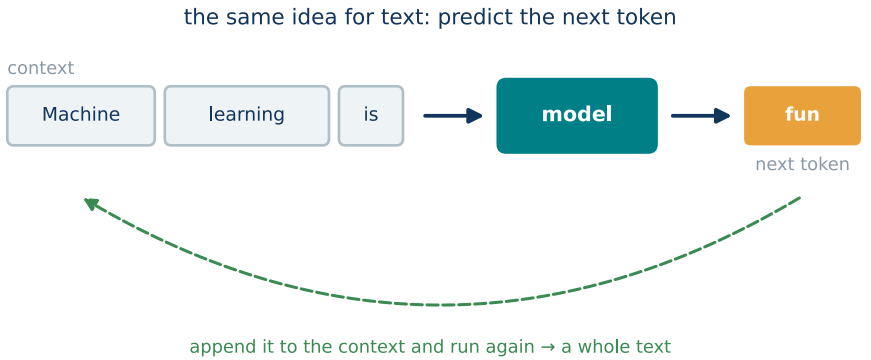
What it is

- The model learns the **distribution** of the data, not a label.
- It can then **draw new samples** — like the training set, but new.

Large language models (LLMs)

- Trained on huge text to **predict the next token**.
- The label is the next word itself — no annotation: **self-supervised**.
- Generating = predict, append, repeat (diffusion models: the same with pixels).





A chat model *answers* instead of *continuing* thanks to a second stage: **instruction tuning** and **RLHF** (reinforcement learning from human feedback). Transformers: the eleventh lecture; generative models: the twelfth.

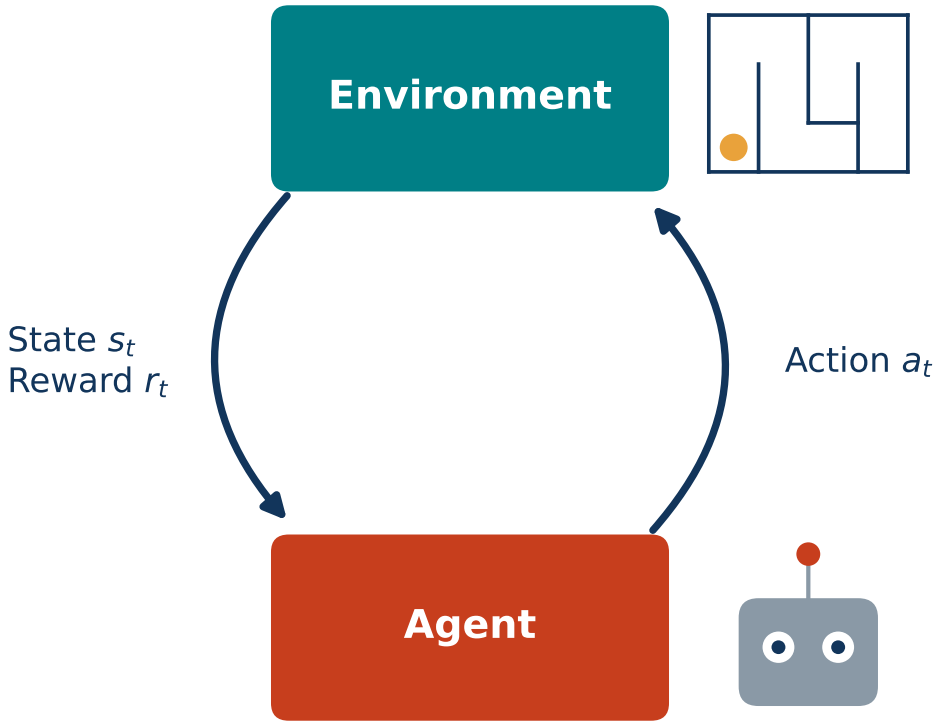
Reinforcement Learning

Definition

- An **agent** interacts with an *environment*: observes the **state**, takes an **action**, receives a **reward**.
- Goal: a policy that maximizes the cumulative reward.

Examples: robots in mazes, AlphaGo, Atari games.

Not in this course — a specific and rather complex field, a course of its own. It returns here only by name: RLHF.



Self-Supervised Learning

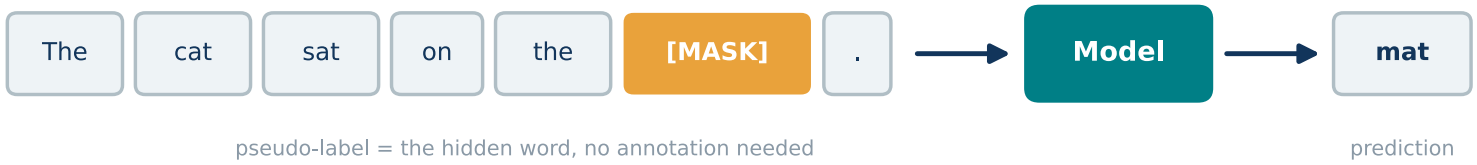
Definition

- **Pseudo-labels** made from the data itself.
- A **pretext task** solved to learn a *representation* for later tasks.

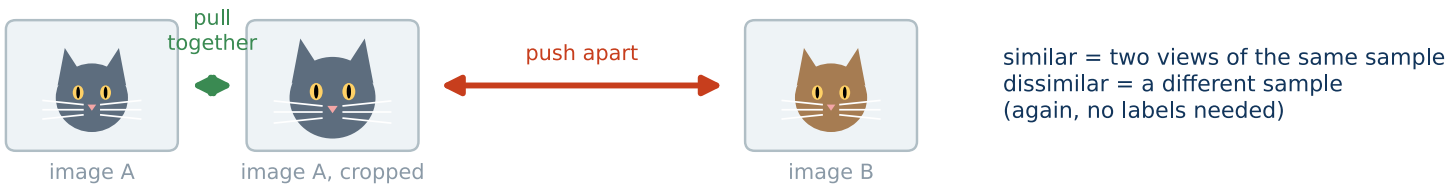
Examples of pretext tasks

- Predict a masked word or missing pixels.
- Contrastive learning: **similar closer**, **dissimilar apart**.
- Next frame in a video, *next word in a sentence (LLMs)*.

Masked-word prediction



Contrastive learning



3 · Optimization

Almost every machine learning method boils down to minimizing something. Let us recall what that means.

Mathematical Background for ML

Three questions, one minute each, on paper:

Calculus

Where is the minimum of $f(x) = 3x^2 + 5x + 1$, and how do you know it is one?

Needed for: derivatives, gradients, the chain rule — and **optimization**, which starts on the next slide.

In ML: training a network is gradient descent on the loss; backpropagation **is** the chain rule.

Linear algebra

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \ ?$$

Needed for: vectors, matrices, eigenvalues.

In ML: a layer of a network is one matrix product $W\mathbf{x}$; PCA is the eigenvectors of the covariance.

Probability and statistics

What is the expected value of one roll of a fair die?

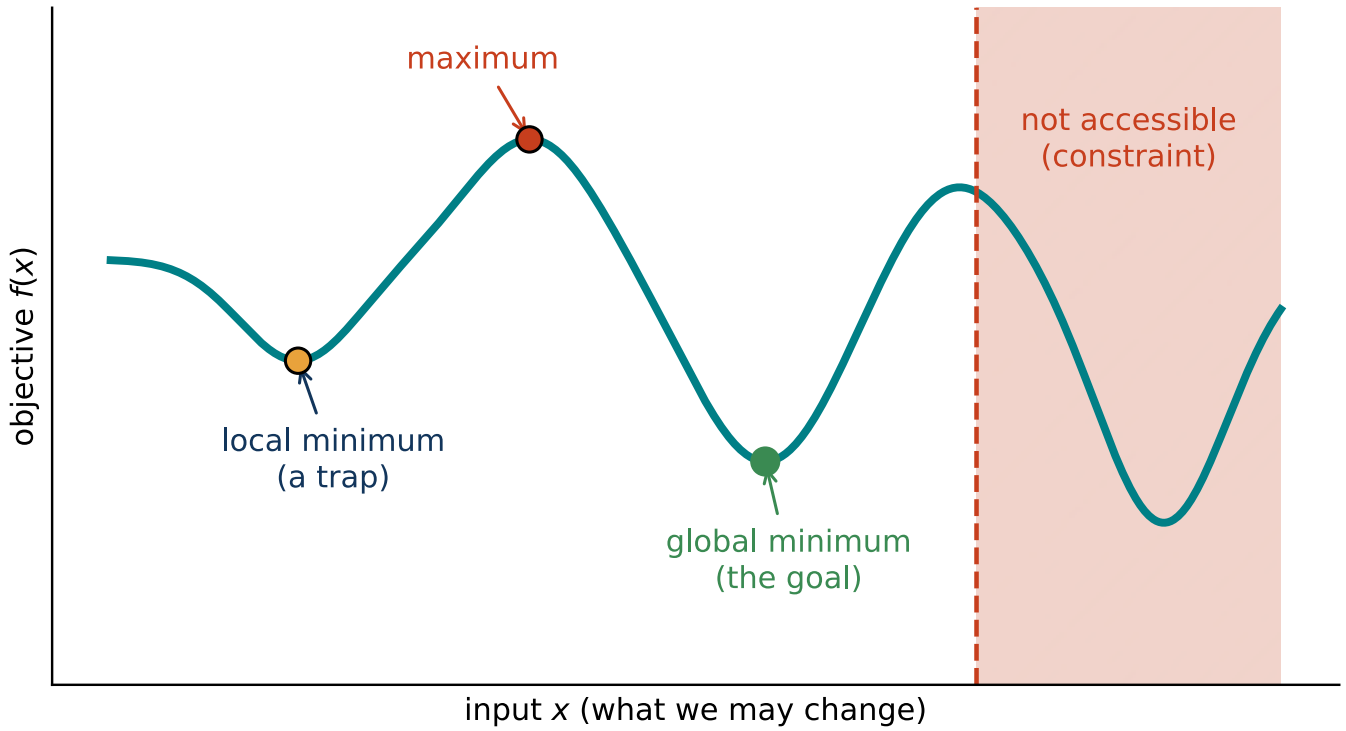
Needed for: random variables, expectation, variance, Bayes' theorem.

In ML: a classifier outputs probabilities and its loss is a log-likelihood; Bayes says what a positive test means for a rare disease.

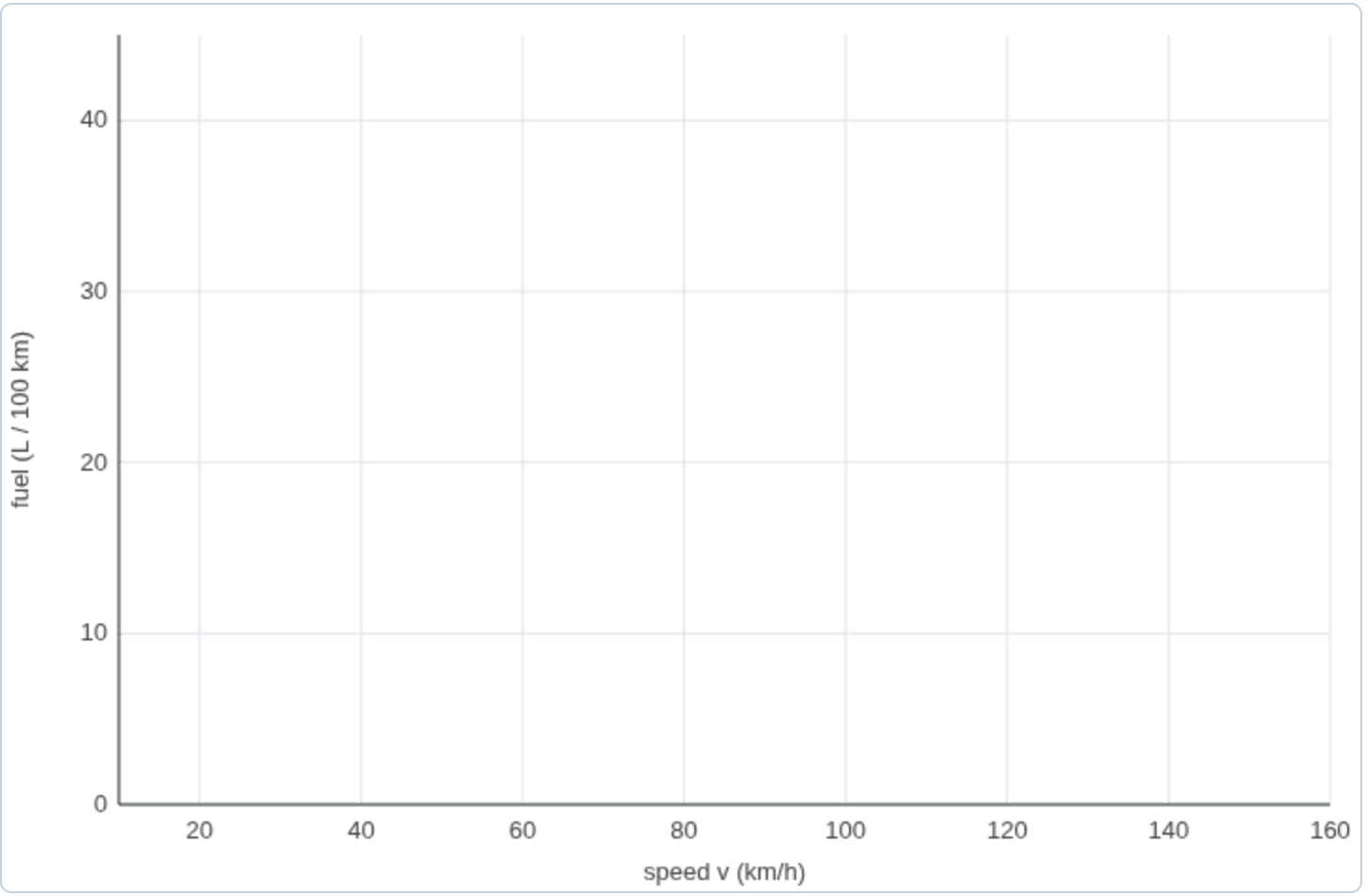
Answers in the recap quiz. If one took you more than a minute, revise it this week. **Optimization** — the fourth area, and the language of every method here — is the rest of today (in depth: MPA-EAL).

Optimization: intuition

- Change the inputs to get the **best possible output**: minimize a cost or maximize a reward.
- A **landscape**: look for the lowest valley.
- Constraints say which parts are **accessible**.
- Almost everything is optimization: a portfolio (max profit), a route (min time)…



Playground: find the most economical speed



Click on a speed to measure the fuel consumption there. The curve is hidden — find the minimum with as few measurements as you can.

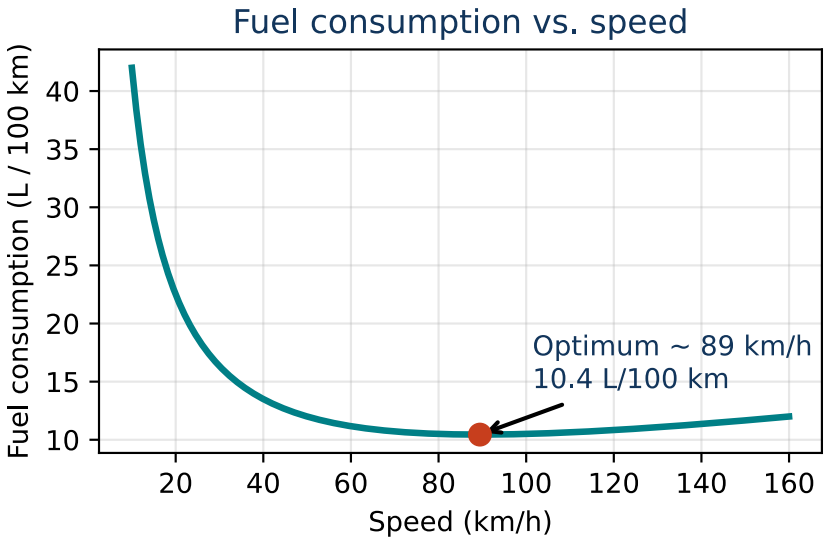
reveal the curve | reset

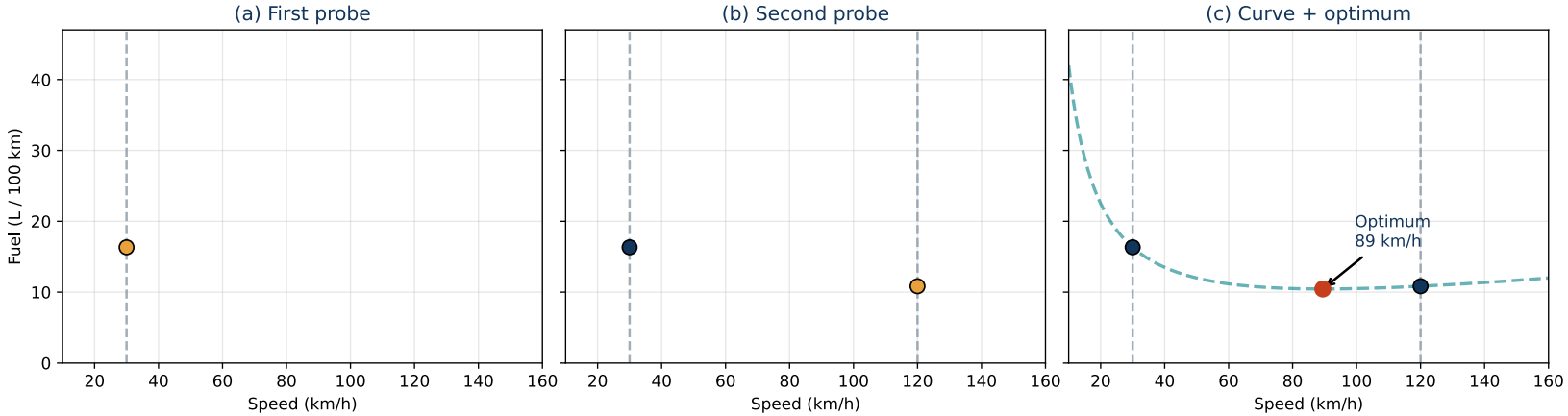
No measurements yet.

Speed vs. fuel (1/3): probing an unknown curve

- Low speed: idling losses, short gearing.
- High speed: aerodynamic drag $\propto v^2$.
- Hence an **optimal speed** in between.
- The true curve is unknown — we can only **probe** it.

One problem, three ways: probing, closed form, gradient descent.





Speed vs. fuel (2/3): the closed form

Simple model:

$$\mathrm{Fuel}(v) = \frac{A}{v} + Bv + C$$

(the rising term is kept linear in v for simplicity)

Optimum condition (derivative equal to zero):

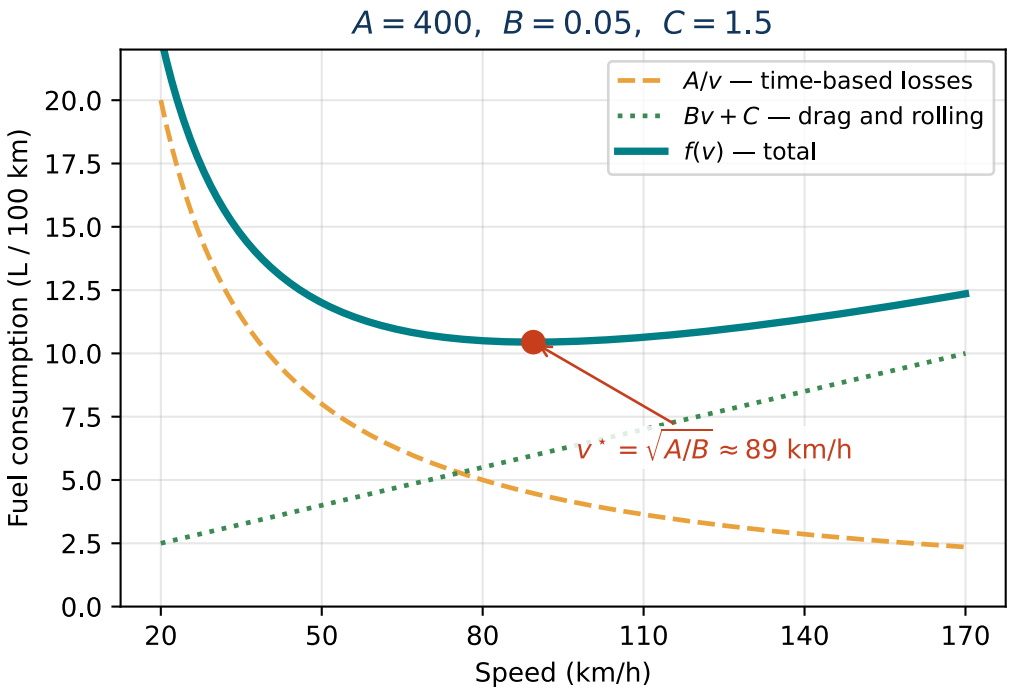
$$\frac{d}{dv}\mathrm{Fuel}(v) = 0 \;\Rightarrow\; -\frac{A}{v^2} + B = 0$$

Solution:

$$v^\star = \sqrt{\frac{A}{B}}$$

Second derivative $\frac{2A}{v^3} > 0$: a **unique minimum**. With $A = 400, B = 0.05$:
$v^\star = \sqrt{8000} \approx 89.4$ km/h, $\mathrm{Fuel}(v^\star) = 2\sqrt{AB} + C \approx 10.4$ L/100 km.

A closed form is a luxury — it needs the formula. Without it: the next slides.



Gradient descent

Intuition: change the input; if the objective improves, keep going, otherwise turn around — as with the speed and the fuel.

The gradient points in the direction of the *steepest increase*; to minimize, we step **against** it.

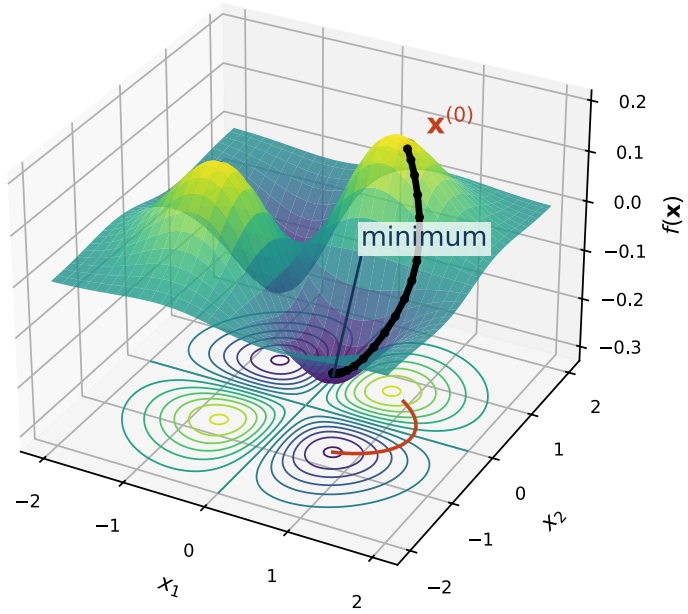
The **gradient** $\nabla f(\mathbf{x})$ is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

Gradient descent update rule:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta \nabla f(\mathbf{x}^{(t)})$$

$\eta > 0$ is the **learning rate** (step size). This is how neural networks are trained.



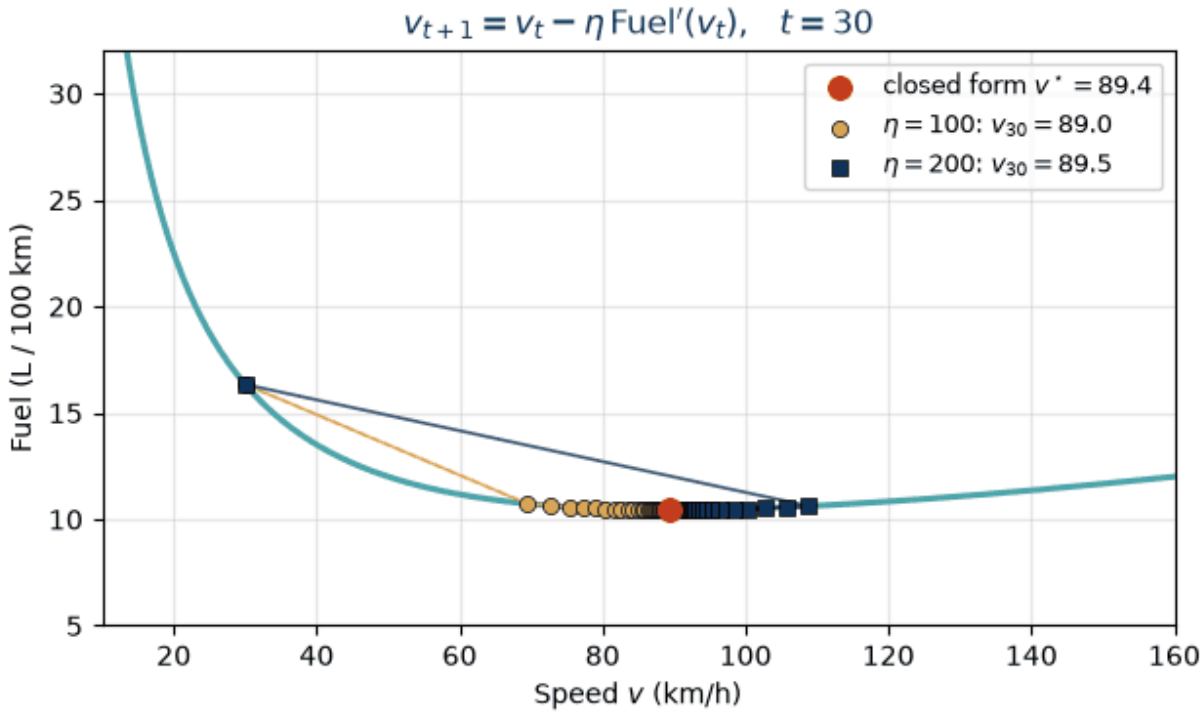
Speed vs. fuel (3/3): gradient descent step by step

One variable, so the gradient is just the derivative:

$$\mathrm{Fuel}'(v) = -\frac{A}{v^2} + B, \qquad v_{t+1} = v_t - \eta\,\mathrm{Fuel}'(v_t)$$

Start at $v_0 = 30$ km/h with $\eta = 200$:

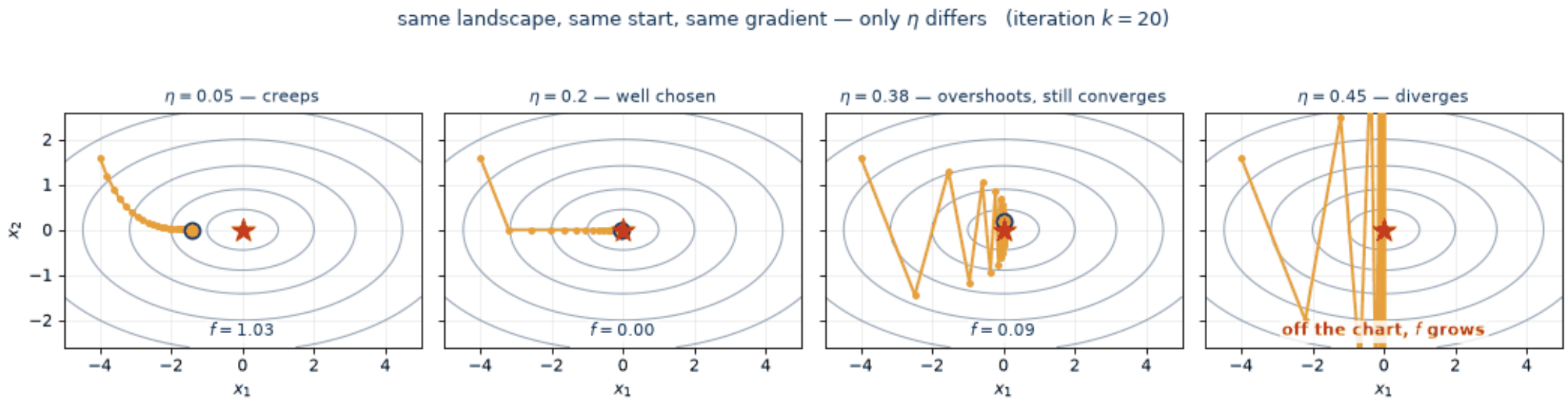
t	v_t	$\mathrm{Fuel}'(v_t)$	v_{t+1}
0	30.0	$-400/900 + 0.05 = -0.394$	108.9
1	108.9	$+0.016$	105.6
2	105.6	$+0.014$	102.8
⋮			
30	89.5	≈ 0	



Both learning rates converge to the closed-form optimum $v^\star = 89.4$ — but a large η overshoots the minimum in the first step, a small one creeps.

Three ways, one problem: probing is blind, the closed form needs the formula, gradient descent needs only the derivative — and that is how neural networks are trained.

Gradient descent: the step size decides everything



On $f(\mathbf{x}) = \frac{1}{2}\left(x_1^2 + 5x_2^2\right)$ the update is $x_i \leftarrow (1 - \eta q_i)\, x_i, q_1 = 1$, $q_2 = 5$: each step multiplies the error by $|1 - \eta q_i|$. Convergence **exactly when**

$$|1 - \eta q_i| < 1 \ \text{ for all } i \qquad \Longleftrightarrow \qquad \eta < \frac{2}{L},$$

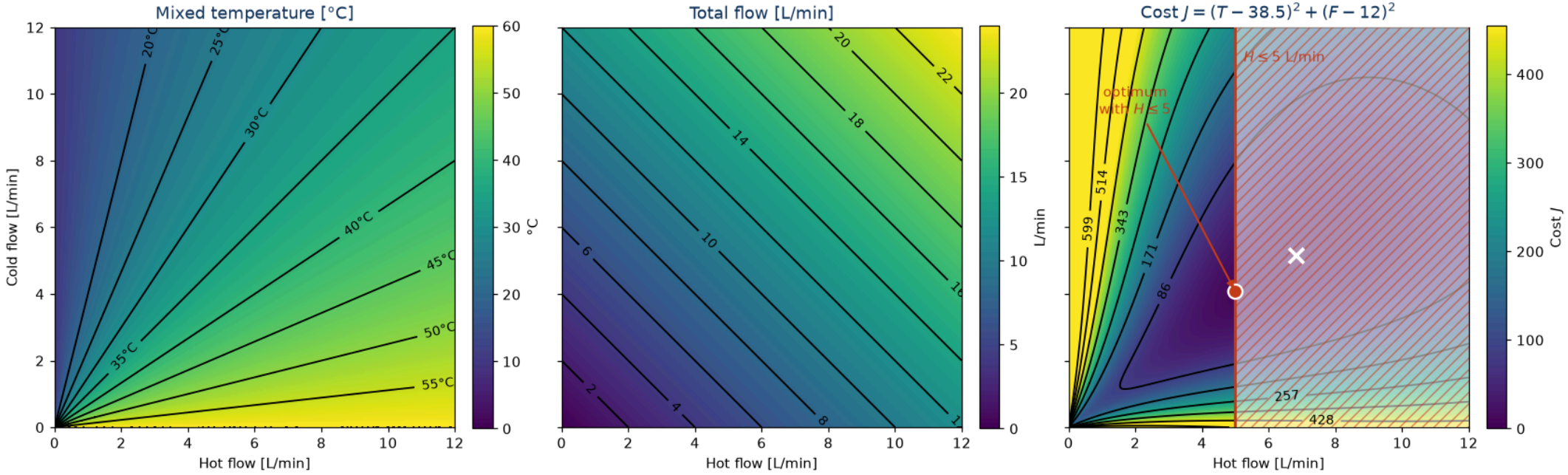
$L = \max_i q_i = 5$ is the largest curvature — hence $\eta < 0.4$ in the panels.

When to stop? Small gradient $\|\nabla f(\mathbf{x})\| < \varepsilon$, small change of f or $\mathbf{x}$, a cap on iterations; when training, the validation error (second lecture).

A small gradient only means a **stationary point** — a saddle or a plateau qualifies too. In a convex problem it is the global minimum.

Optimization example 2D: mixing hot and cold water

Mixing cold and hot water is an optimization task. We need to set a comfortable flow **and** a comfortable temperature.



Hot 60 °C, cold 10 °C, target 38.5 °C at 12 L/min: $H + C = 12,\ \frac{60H+10C}{12} = 38.5 \Rightarrow H = 6.84,\ C = 5.16$ (the white cross). **Boiler limited to** $H \leqslant 5$ (hatched): the optimum moves to the boundary, $C = 4.08$ — 37.5 °C, 9.1 L/min, $J = 9.5$ instead of 0.

Definition of optimization

Problem definition:

$$\mathbf{x}^\star \in \arg\min_{\mathbf{x}\in X} f(\mathbf{x})$$

Canonical form:

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \le 0, \quad i = 1,\dots,m \\ & h_j(\mathbf{x}) = 0, \quad j = 1,\dots,p \end{aligned}$$

$$X = \{\mathbf{x} \in \mathbb{R}^n : g_i(\mathbf{x}) \le 0,\ h_j(\mathbf{x}) = 0\}$$

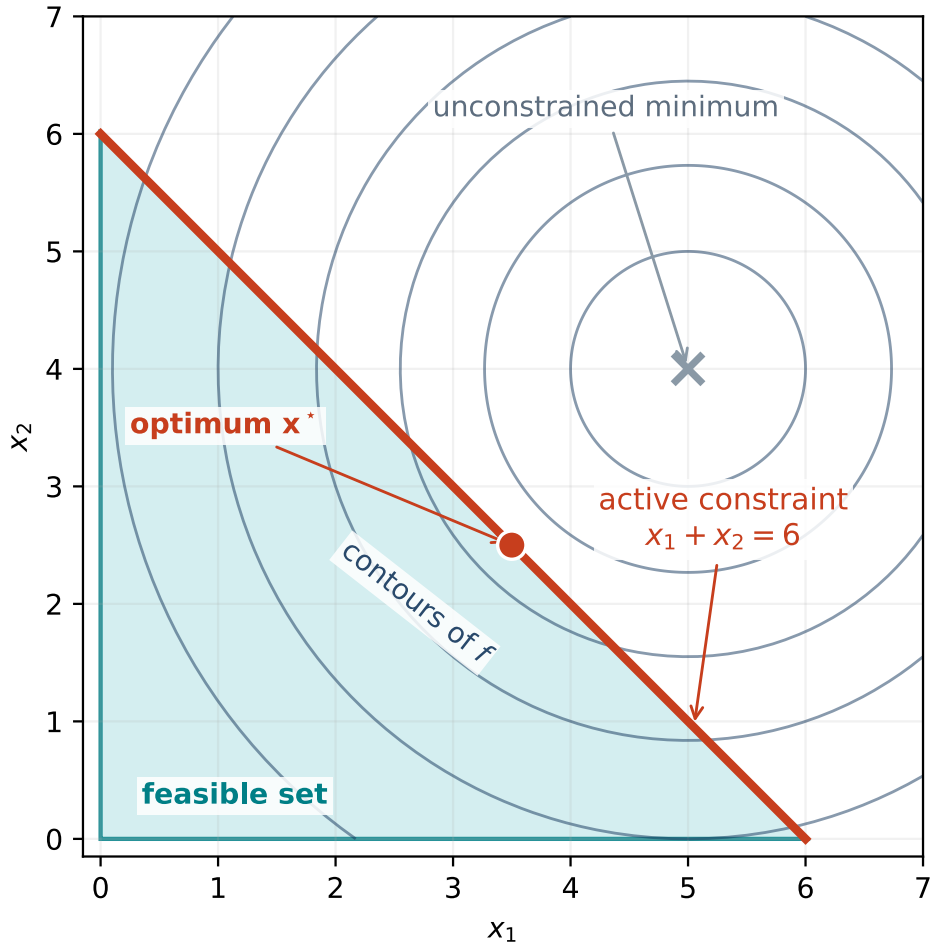
Every problem has the same three parts:

- **variables** $\mathbf{x}$ — what we may change (the speed, the taps, the weights);
- **objective** $f(\mathbf{x})$ — the one number we push down;
- **constraints** g_i, h_j — they carve out the **feasible set** X.

Minimum ≠ minimizer: $p^\star = \min f$ is a *number*, $\mathbf{x}^\star$ a *point* — possibly one of many.

Convention: we always minimize. $\max_{\mathbf{x}} u(\mathbf{x})$ becomes $\min_{\mathbf{x}} -u(\mathbf{x})$: same minimizer, opposite value.

Feasible set and active constraints



- **Feasible** = satisfies all constraints.
- If the **unconstrained** minimum is infeasible, the optimum is pushed onto the boundary.
- A constraint is **active** at $\mathbf{x}^\star$ when it holds with **equality** — it is what stops the optimum.
- Here $x_1 + x_2 \leq 6$ is active, $x_1 \geq 0$ and $x_2 \geq 0$ are not.

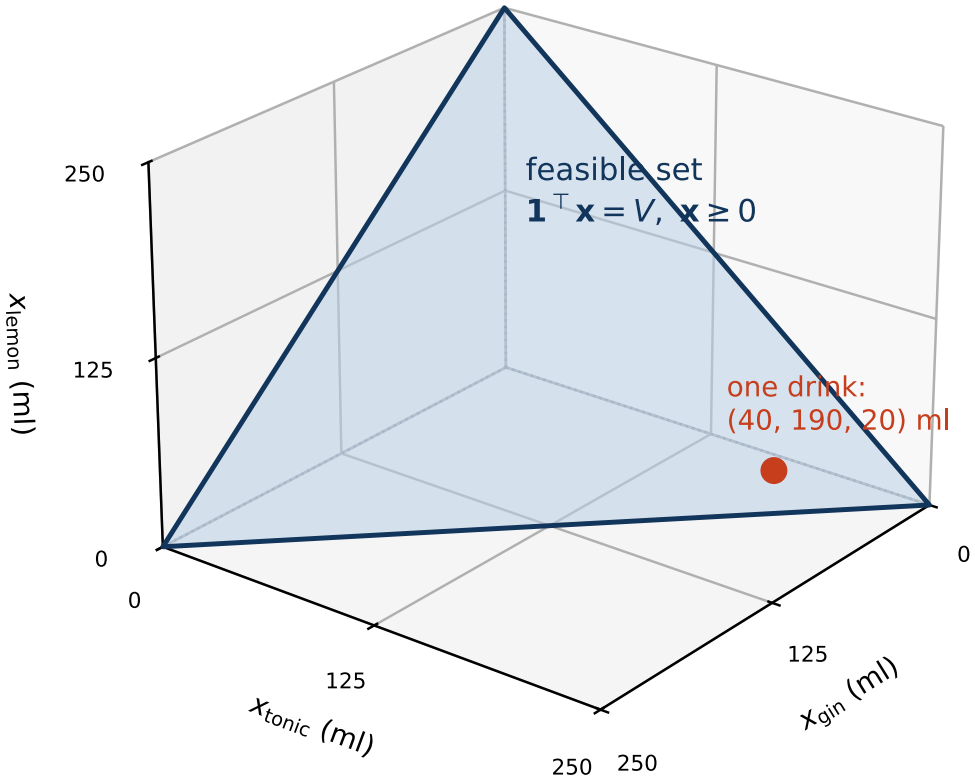
A generic example, not the tap. The tap with $H \leq 5$: **active**. The fuel curve with $50 \leq v \leq 130$ km/h: **inactive**, $v^\star = 89.4$ lies inside.

Constrained optimization: a Gin & Tonic

Task: mix gin, tonic and lemon juice (ml), $\mathbf{x} = (x_{\text{gin}}, x_{\text{tonic}}, x_{\text{lemon}})^\top$, so that the **glass is full** and the taste $T(\mathbf{x})$ is best:

$$\begin{aligned} \text{minimize} \quad & -T(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{1}^\top \mathbf{x} = V \quad \text{(glass is full)} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

The feasible set is the triangle on the right ($V = 250$ ml).



$T(\mathbf{x})$ has no formula — its value comes from a tasting. Such problems are **black-box**; tuning hyperparameters is one of them.

Hard constraint or soft preference?

Constraint — “this must not be violated”

$$g_i(\mathbf{x}) \leq 0$$

A safety limit, a budget, a law of physics. It can make the problem **infeasible**.

Objective term — “the less of it the better”

$$f(\mathbf{x}) = \text{error} + \lambda \cdot \text{penalty}$$

A preference; λ says how much we care. It only shifts the optimum.

Either way is possible and the answers differ. Ridge regression (fifth lecture): the budget $\|\mathbf{w}\|_2^2 \leq t$ becomes the penalty $\lambda\|\mathbf{w}\|_2^2$.

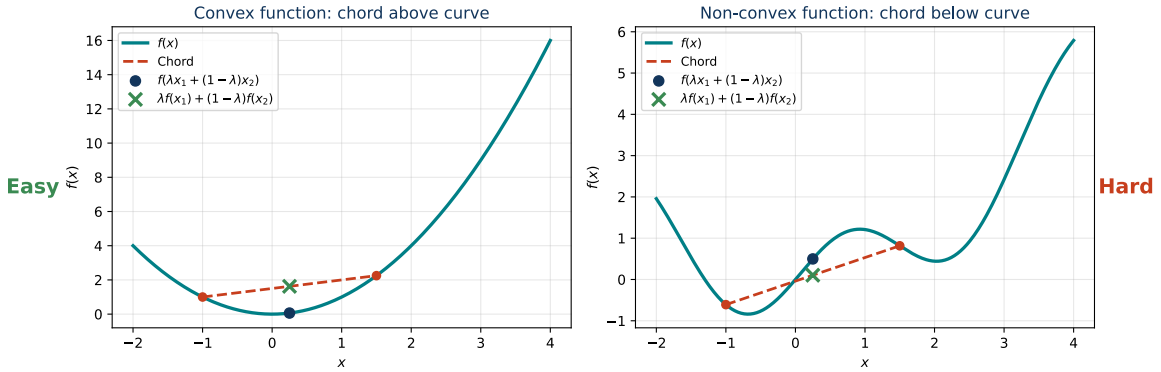
The solver optimizes what we wrote down, not what we meant. Minimize the training error alone and the model memorizes the data; maximize sensitivity alone and everybody is ill.

Convexity: functions and sets

Convex function: for all $\mathbf{x}_1, \mathbf{x}_2$ and $\lambda \in [0, 1]$

$$f(\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) \;\leq\; \lambda f(\mathbf{x}_1) + (1-\lambda) f(\mathbf{x}_2)$$

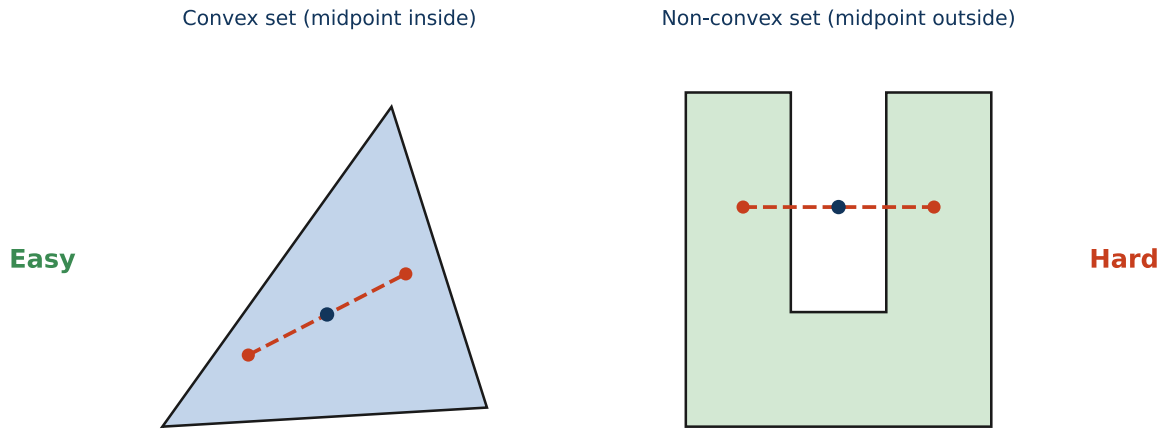
The graph lies **below any chord**; no local minima other than the global one.



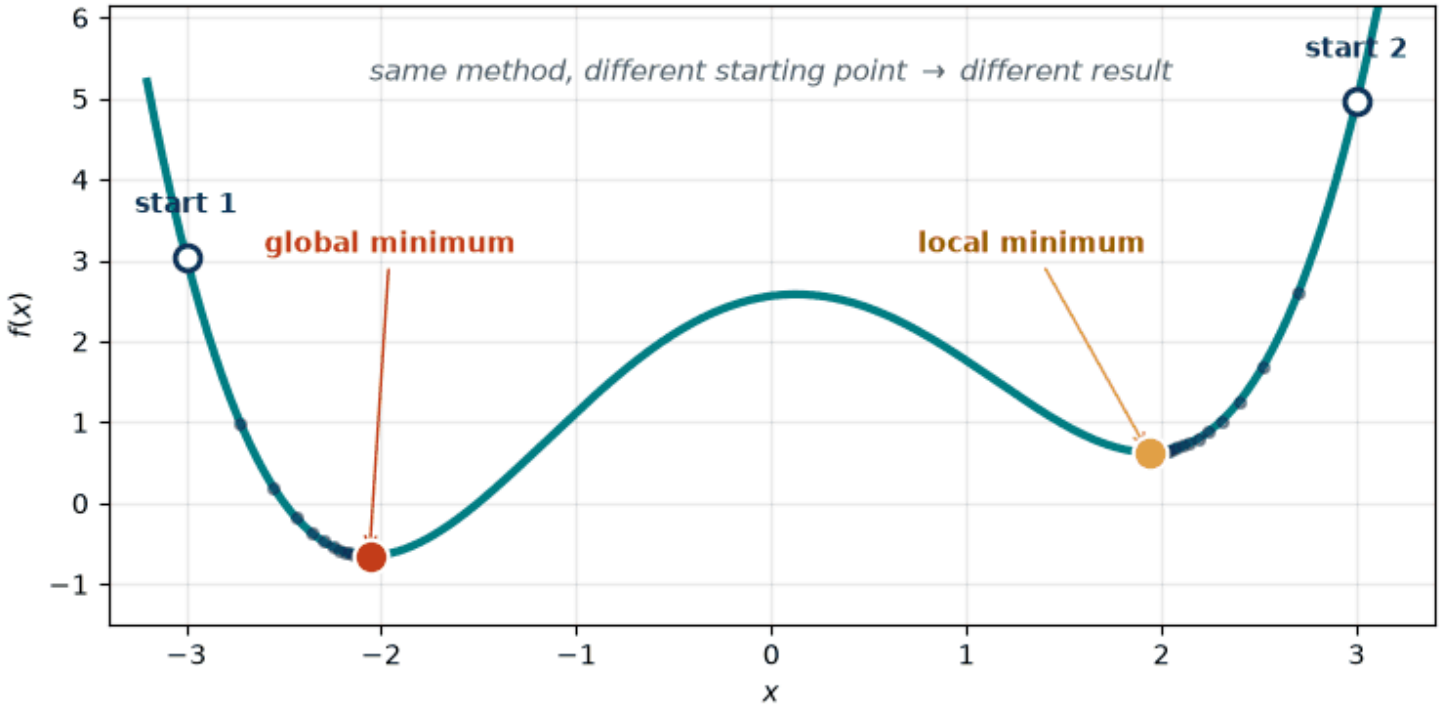
Convex set: for all $\mathbf{x}_1, \mathbf{x}_2 \in C$ and $\lambda \in [0, 1]$

$$\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2 \in C$$

The **segment** between any two points of C stays in C; no barrier hides the minimum.

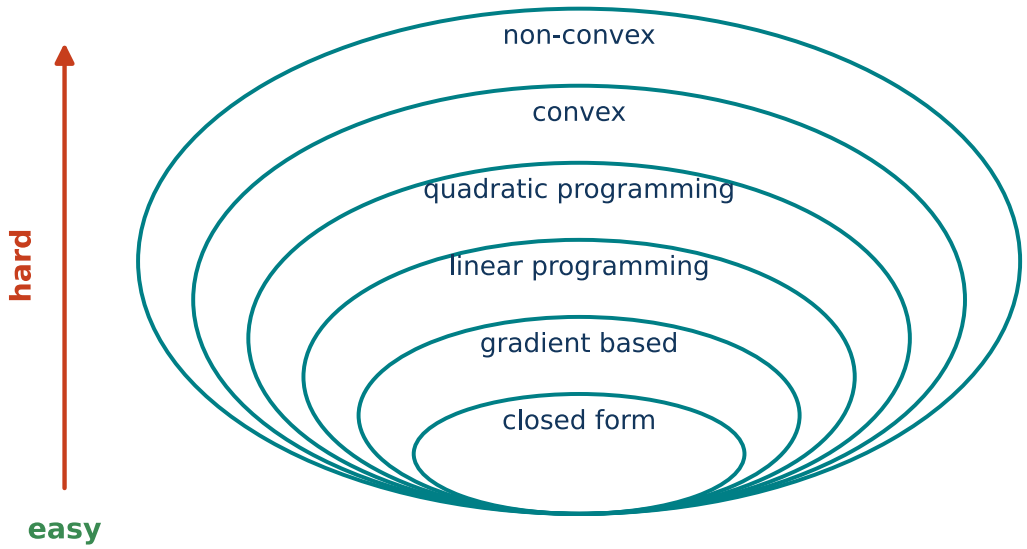


Local vs. global minimum — and what convexity buys



- **Local minimum:** best in a neighbourhood; **global:** best in the whole feasible set.
- A local method ends in the valley of its **starting point** — and cannot tell.
- **Convex problem** (convex f over a convex set): **every local minimum is global**.
- Not guaranteed: that a minimizer **exists** ($\min_x e^x$), that it is **unique**, that the model is right.

Optimization approaches



- *Closed form* — a formula, no iterations.
- *Gradient-based* — the derivative says where to go.
- *Linear programming* — linear f and constraints; fast solvers.
- *Quadratic programming* — quadratic f, linear constraints; fast solvers.
- *Convex* — easy, every local minimum is global.
- *Non-convex* — hard; heuristics without guarantees.
- *Discrete* — outside the diagram; usually NP-hard.

The first two name a **method**, the rest a **property of the problem** — and the problem picks the method.

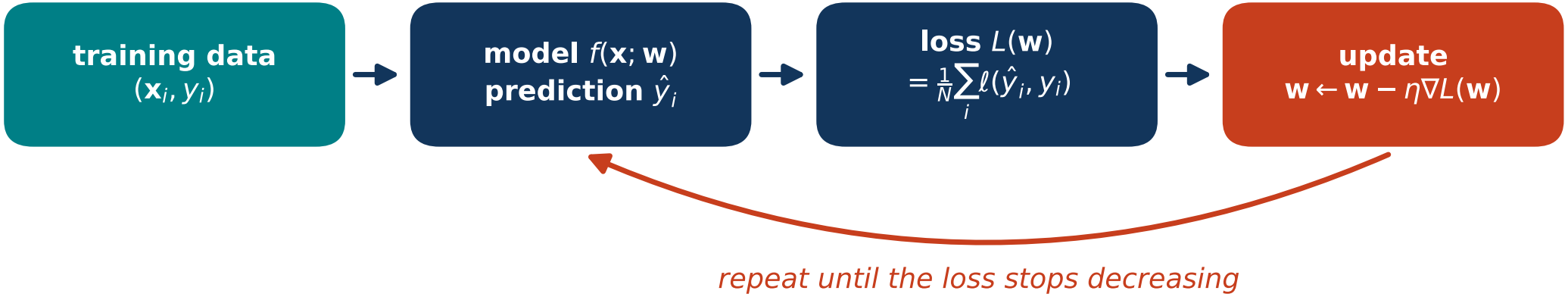
Optimization approaches: examples

Where each class shows up in this course:

- *Closed form:* ridge regression (a matrix inverse), the sample mean and variance, PCA (eigenvectors).
- *Gradient-based:* logistic regression, neural networks.
- *Linear programming:* L_1 regression (least absolute deviations).
- *Quadratic programming:* soft-margin SVM.
- *Convex:* lasso regression, logistic regression.
- *Non-convex:* deep neural networks, k-means clustering, tuning hyperparameters (a black box, like the Gin & Tonic).
- *Discrete optimization:* feature selection.

Training a model is optimization

training a model = minimizing $L(\mathbf{w})$ over the weights w



- **Variables** = the parameters $\mathbf{w}$, not the data: a line has 2, a small network 10^6, an LLM 10^{11}.
- **Objective** = the loss averaged over the training set.
- Usually **no constraints** — plain gradient descent is enough.

- No formula for $\mathbf{w}^\star$ — but the loss and its gradient at a point are all descent needs.
- From the seventh lecture on, every network is trained by this loop.
- Even k-means is this: an objective, a bad landscape, a heuristic.

Recap Quiz (1/2)

- **Classical programming vs. ML:** what are the inputs and outputs in each paradigm?
- **AI vs. ML vs. DL:** which is the umbrella term and how do ML and DL differ?
- **Narrow AI vs. AGI:** which one is everything in this course, and what would the other one be able to do?
- **Branches:** give one task typical for *supervised*, *unsupervised*, *reinforcement* and *self-supervised* learning.
- **Regression vs. classification:** what is predicted in each case?
- **Optimization tasks:** define an everyday task and an image or signal processing task as an optimization problem — name the variables, the objective and the constraints.
- **Minimum vs. minimizer:** which of the two is a number and which one is a point?

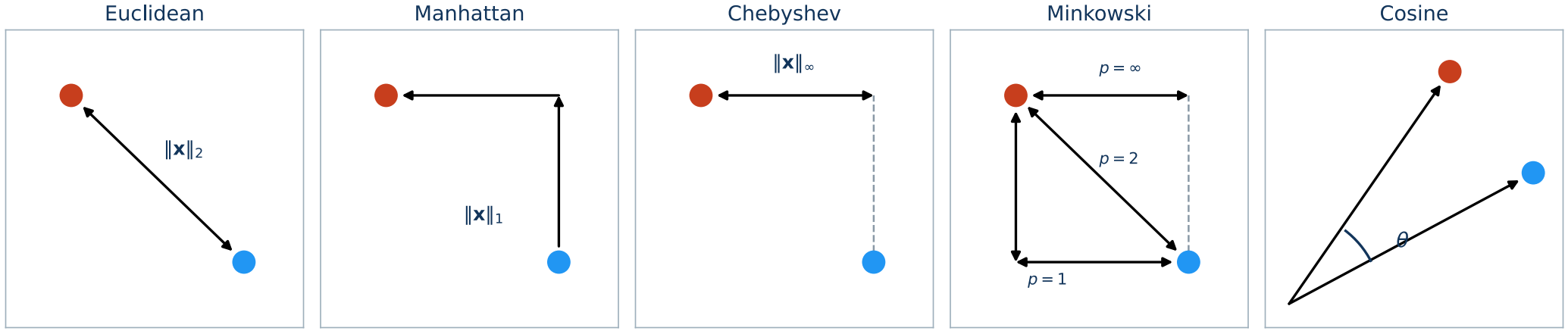
Recap Quiz (2/2)

- **Feasible set:** when is a constraint *active* at the optimum, and what does that tell us?
- **Convexity:** name one property of convex functions/sets that simplifies optimization — and one thing convexity does *not* guarantee.
- **Gradient & GD:** define $\nabla f(\mathbf{x})$ — scalar or vector, how many components? — and write one update step.
- **GD:** what happens when the learning rate is too large, and what sets the limit?
- **The three warm-up questions:** $x^\star = -5/6$ (and $f'' = 6 > 0$); $(1 \cdot 1 + 2 \cdot (-1),\ 3 \cdot 1 + 4 \cdot (-1)) = (-1, -1)$; $(1 + 2 + \cdots + 6)/6 = 3.5$.

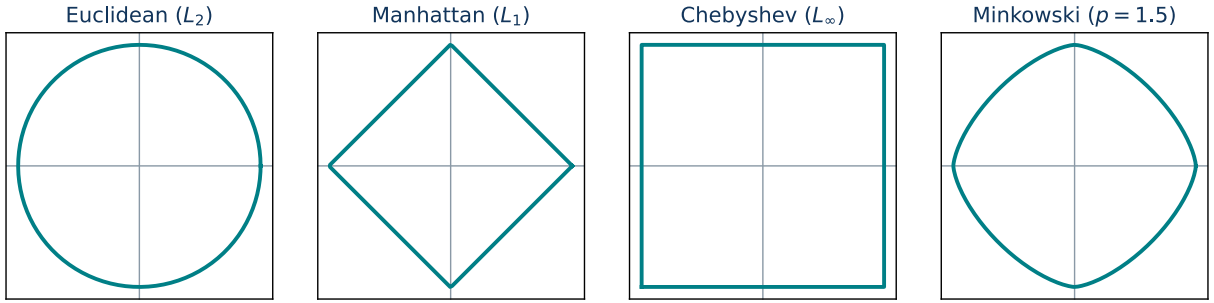
Appendix · Recap of BPC-UIN algorithms

Distances, nearest neighbours, k-means and UPGMA — a refresher from the bachelor course. Not presented in the lecture: read it before the first lab, which computes distances and a nearest-neighbour decision by hand.

Distances and vector norms



Unit balls



$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2} \quad \|\mathbf{x}\|_1 = \sum_i |x_i|$$

$$\|\mathbf{x}\|_\infty = \max_i |x_i| \quad \|\mathbf{x}\|_p = \left(\sum_i |x_i|^p\right)^{1/p}$$

For a distance, let $\mathbf{x} = \mathbf{a} - \mathbf{b}$ be the difference vector.

Example: $\mathbf{a} = (1, 1), \mathbf{b} = (2, 3)$, so $\mathbf{x} = (-1, -2)$:
$\|\mathbf{x}\|_2 = \sqrt{5} \approx 2.24, \|\mathbf{x}\|_1 = 3, \|\mathbf{x}\|_\infty = 2.$

code example

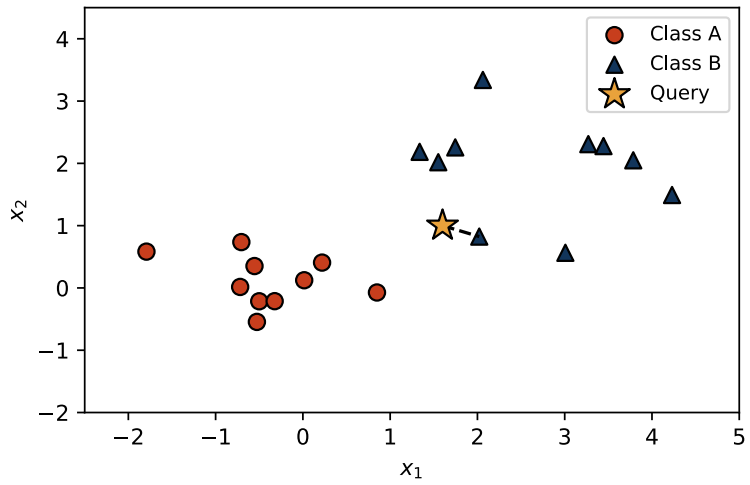
Nearest Neighbor (1-NN)

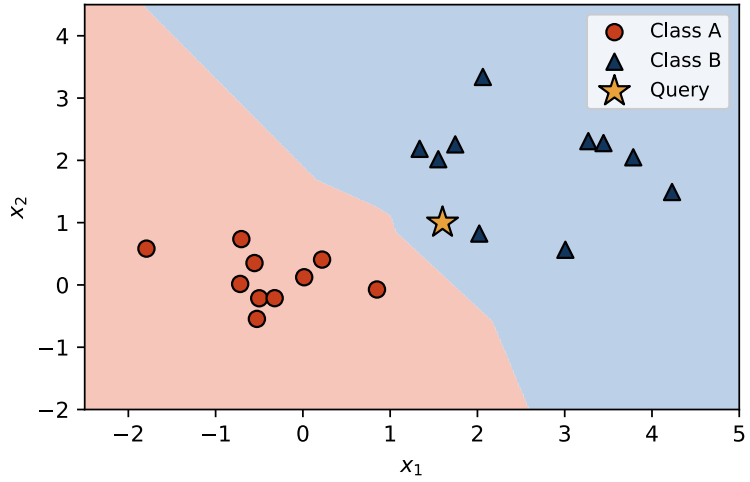
Idea

- For $\mathbf{x}_*$ find the *closest* training point and copy its label: $\hat{y}(\mathbf{x}_*) = y_{(1)}$.

Properties

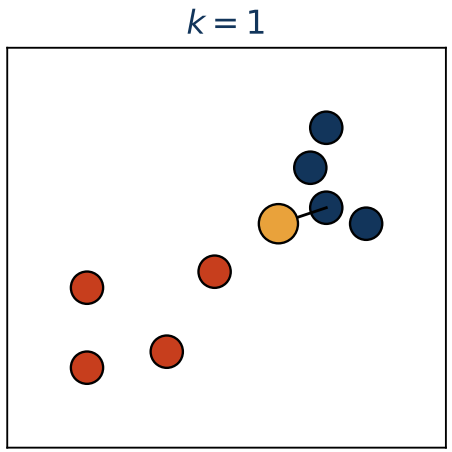
- No training phase (lazy learning).
- Sensitive to noise, scaling and outliers.
- Search is expensive for large n (kd-tree).

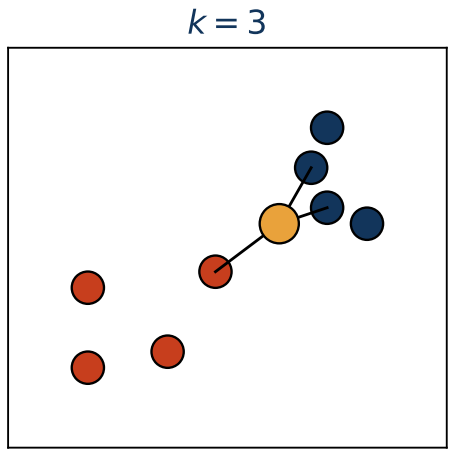


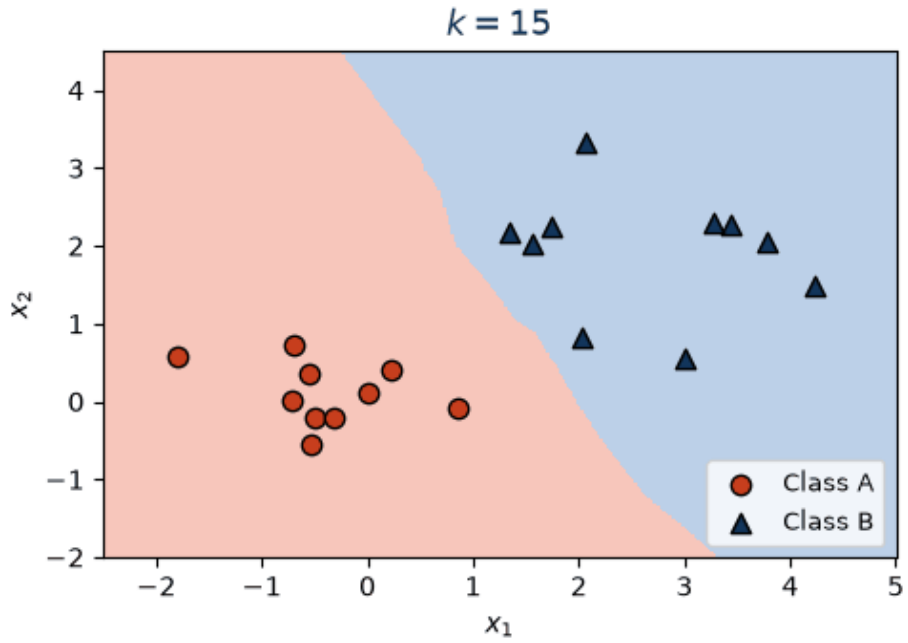


k-Nearest Neighbors (k-NN)

- Find the k closest neighbors $\mathcal{N}_k(\mathbf{x}_*)$; majority vote (classification) or average (regression).
- k regularizes the model (bias–variance trade-off).

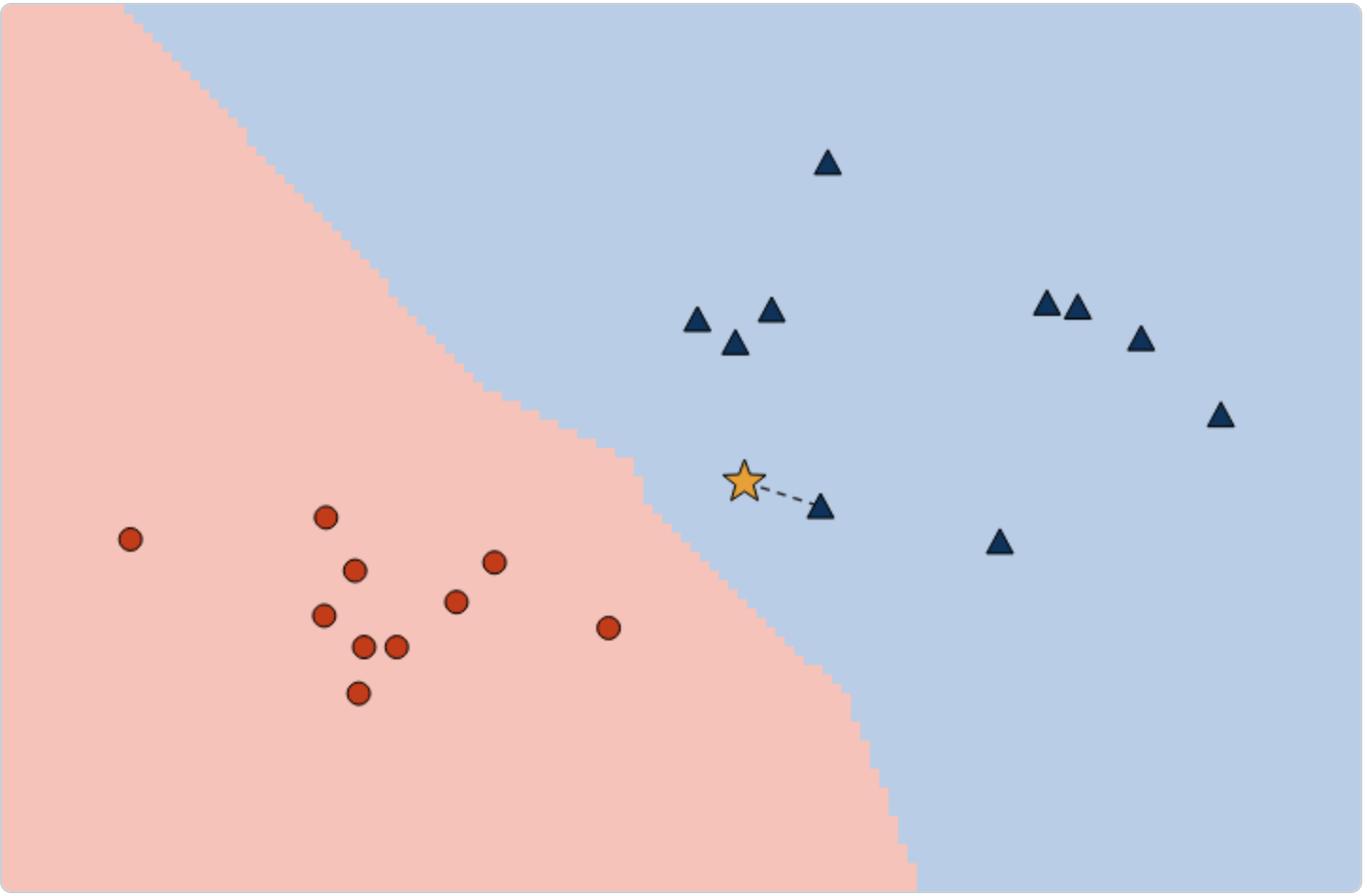






Playground on the next slide · code example

k-NN playground



k = 1

Metric: Euclidean (L2)

Add on click: A (red circle) B (blue triangle)

Click on the canvas to add a point (**right-click** or **shift-click** always adds B), **drag** any point or the orange query star to move it.

reset

Query (1.60, 1.00): neighbours **0 × A, 1 × B** → class **B**. Points: 20.

The twenty points are those of the 1-NN slide; the shading is the classifier's decision. Odd k only, so the vote cannot tie.

k-means Clustering

Idea

- n points into k clusters, each represented by its mean.
- Minimize the **inertia**:

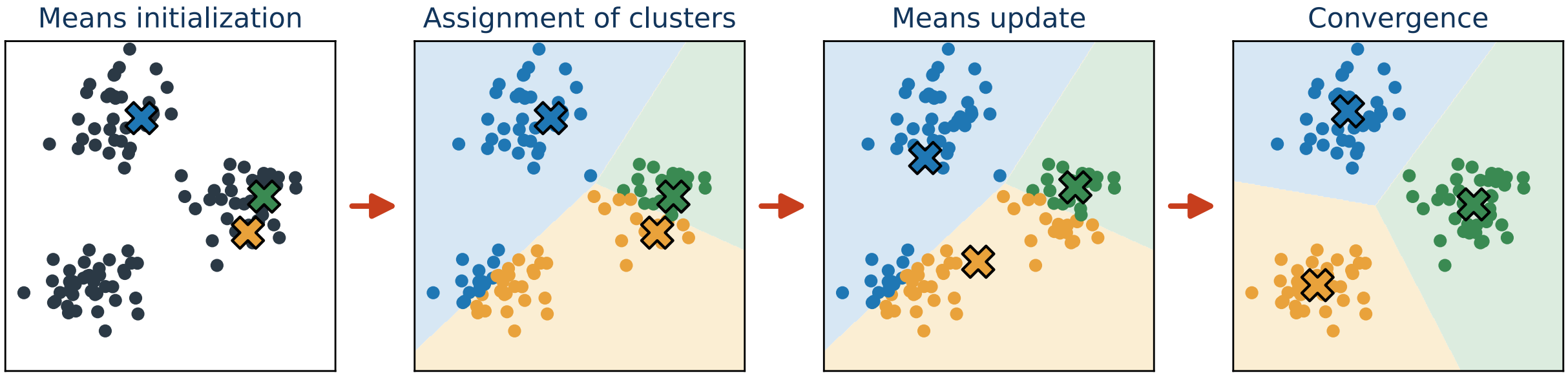
$$J = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2$$

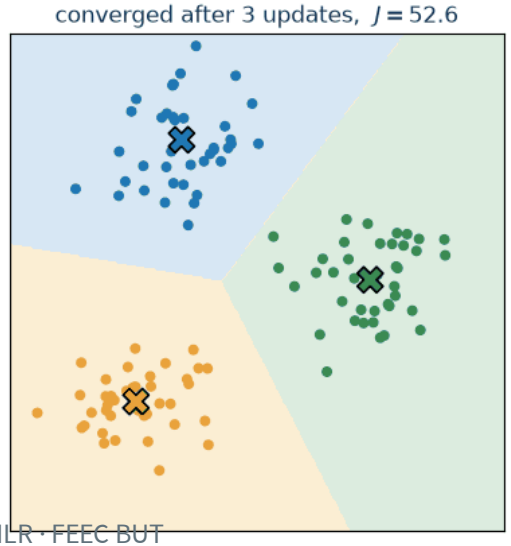
Algorithm (Lloyd's)

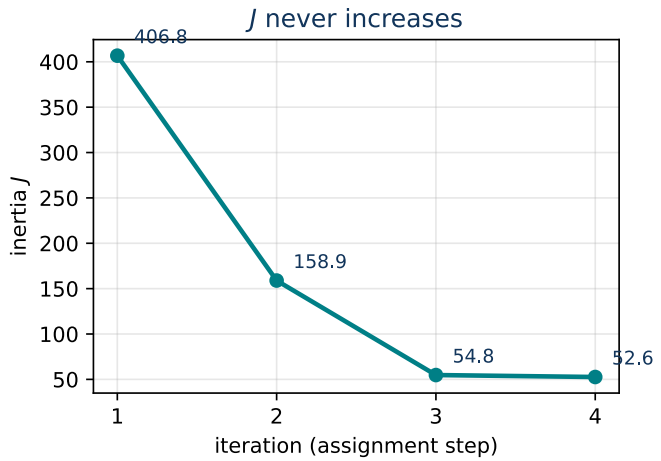
1. Initialize k centroids (random samples).
2. Assign each $\mathbf{x}_i$ to the nearest centroid.
3. Move centroids to the means of their points.
4. Repeat 2–3 until nothing changes.

- Depends on initialization (restart several times); finds a local minimum.

k-means Clustering: the algorithm step by step







Every assignment step and every means update can only **decrease** J — that is why Lloyd's algorithm always converges (to a local minimum).

code example

By hand: a k-NN vote and one k-means step

Six training points — the data of `code/knn.py`: class A $(1, 2)$, $(1.5, 1.8)$, $(1, 0.6)$; class B $(5, 8)$, $(8, 8)$, $(9, 11)$.

k-NN, $k = 3$, new sample $(2, 3)$

point	class	d_2 to $(2, 3)$
$(1.5, 1.8)$	A	**1.30**
$(1, 2)$	A	**1.41**
$(1, 0.6)$	A	**2.60**
$(5, 8)$	B	5.83
$(8, 8)$	B	7.81
$(9, 11)$	B	10.63

Three nearest: A, A, A $\Rightarrow$ **class A** ($k = 5$: 3 : 2, still A).

k-means, $k = 2$, initial means $\boldsymbol{\mu}_1 = (1, 2)$, $\boldsymbol{\mu}_2 = (5, 8)$

point	d to $\boldsymbol{\mu}_1$	d to $\boldsymbol{\mu}_2$	cluster
$(1, 2)$	0.00	7.21	1
$(1.5, 1.8)$	0.54	7.12	1
$(5, 8)$	7.21	0.00	2
$(8, 8)$	9.22	3.00	2
$(1, 0.6)$	1.40	8.41	1
$(9, 11)$	12.04	5.00	2

Assignment: $J = 36.25$. Update: $\boldsymbol{\mu}_1 = (1.17,\ 1.47)$, $\boldsymbol{\mu}_2 = (7.33,\ 9.00)$, $J = 15.98$. Next assignment unchanged — **converged**.

UPGMA

Unweighted Pair Group Method With Arithmetic Mean

Idea

- Hierarchical: repeatedly merge the two closest clusters.
- Cluster distance = average of all pairwise distances.
- Result: a tree (dendrogram).

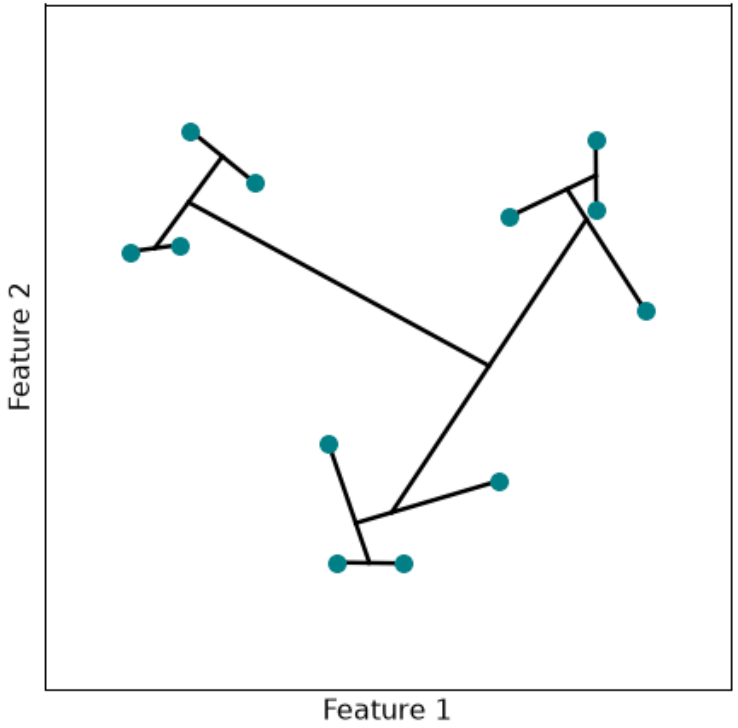
Algorithm

1. Every sample is a cluster.
2. Merge the closest pair.
3. Update the distance matrix (averages).
4. Repeat until one cluster remains.

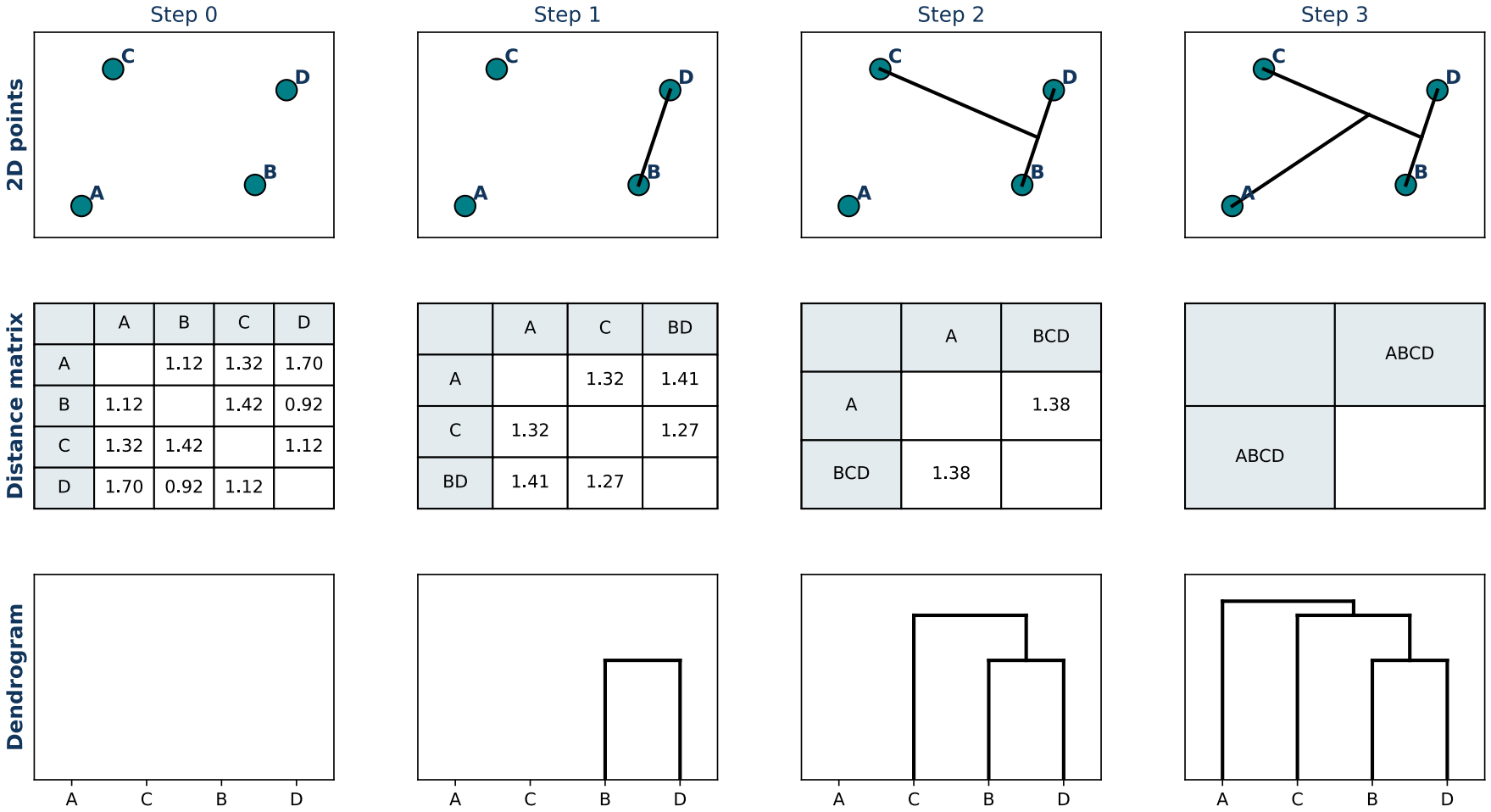
Cluster distance definition

Merging C_i and C_j ($|C_i|$ = size): the distance to another cluster C_k becomes

$$d(C_{i\cup j}, C_k) = \frac{|C_i|\, d(C_i, C_k) + |C_j|\, d(C_j, C_k)}{|C_i| + |C_j|}$$



UPGMA: practical construction



code example

Appendix: check yourself

- **Distances:** write the formulas for $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_1$ and $\|\mathbf{x}\|_\infty$ (for $\mathbf{x} = \mathbf{a} - \mathbf{b}$).
- **1-NN vs. k-NN:** how does the prediction differ and what is k trading off?
- **k-means:** what objective does it minimize and what makes it sensitive?
- **UPGMA:** how is the distance to a newly merged cluster computed?